https://hpcleuven.github.io/Linux-tools/



Linux tools

Mag Selwa, Geert Jan Bex

SCAN ME

Overview

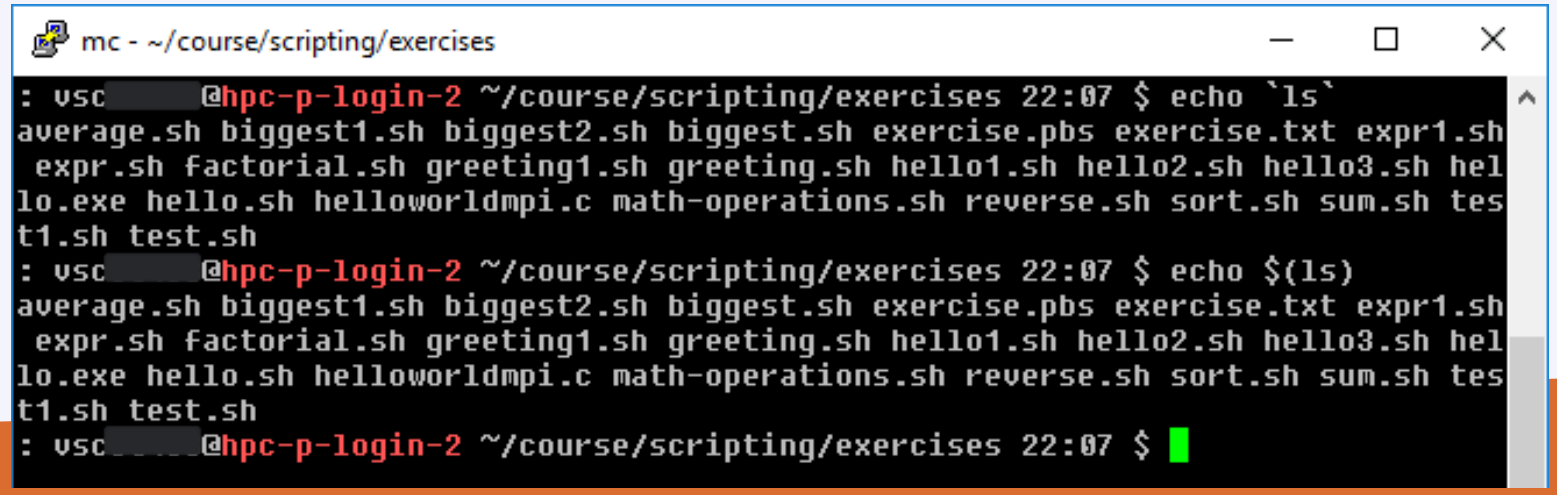
- Few important issues: quotes, pipes, redirections, output
- Some useful commands
- Grep
- Sed
- Awk
- Gnuplot
- ImageMagick

Objectives

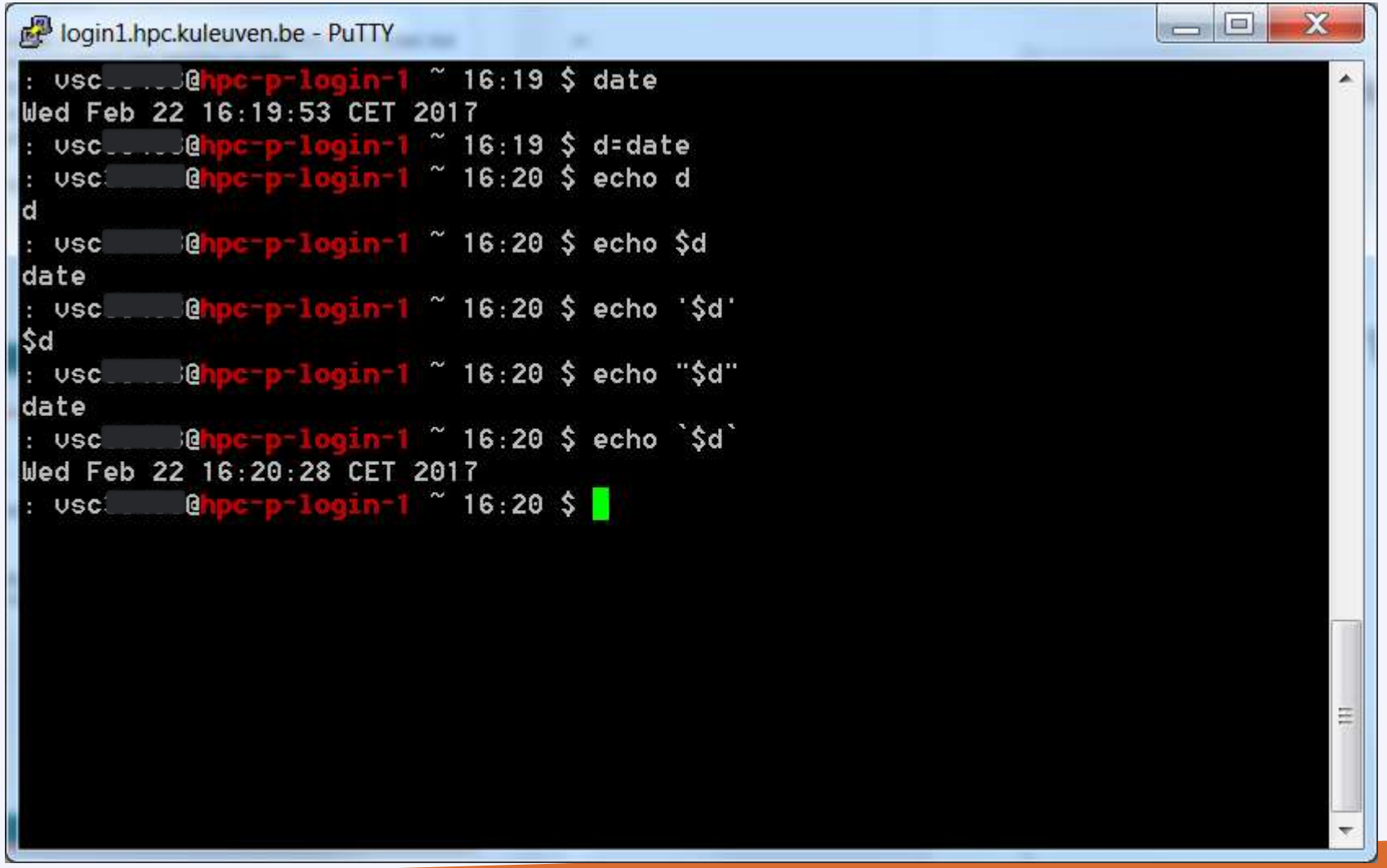
- The material in this lecture is **not a complete** guide or manual.
- The purpose is to get overview of the capabilities of the different tools and to underline their particular strengths and disadvantages.
- The course aims to promote the tools for use in your everyday research work and urges you find solutions yourself rather than expecting a complete solution.

Difference Between Single, Double, and Backwards Quote

- Single quotes (‘) do not interpret any variables
- Double quotes (“) interpret variables
- Backwards quotes (`) interpret variables and treat them as a program to run and return the results of that program
- These two operators do the same thing. Compare these two lines: `` vs $()



Example Of Quote Difference



<() vs $()

- <() is similar to $() in that the output of the command inside is re-used.
- In this case, though, the output is treated as a file. This file can be used as an argument to commands that take files as an argument.
- Try:

```
$ grep somestring file1 > /tmp/a
$ grep somestring file2 > /tmp/b
$ diff /tmp/a /tmp/b
```

- instead you can write:

```
$ diff <(grep somestring file1) <(grep somestring file2)
```

Quote characters

There are three different quote characters with different behaviour. These are:

“ : double quote, weak quote. If a string is enclosed in “ ” the references to variables (i.e *$variable*) are replaced by their values. Also back-quote and escape \ characters are treated specially.

‘ : single quote, strong quote. Everything inside single quotes are taken literally, nothing is treated as special.

` : back quote. A string enclosed as such is treated as a command and the shell attempts to execute it. If the execution is successful the primary output from the command replaces the string.

Example: `echo “Today is:” `date``

Input and Output

- Programs and commands can contain an input and output. These are called streams. UNIX programming is oftentimes stream based.

- STDIN – 'standard input,' or input from the keyboard
- SDTOUT – 'standard output,' or output to the screen
- STDERR – 'standard error,' error output which is sent to the screen.

File Redirection

- Often, we want to save output (stdout) from a program to a file. This can be done with the 'redirection' operator.
  - **myprogram** > *myfile* – using the '>' operator we redirect the output from **myprogram** to file *myfile*

- Similarly, we can append the output to a file instead of rewriting it with a double '>>'
  - **myprogram** >> *myfile* – using the '>' operator we append the output from **myprogram** to file *myfile*

Input Redirection

- Input can also be given to a command from a file instead of typing it to the screen, which would be impractical.
  - mycommand < *programinput* – Here **mycommand** gets its input from *programinput* instead of waiting for what you type.

Redirecting stderr

- Performing a normal redirection will not redirect sdterr. In Bash, this can be accomplished with ‘2>’
  - **command** *2> file1*

- Or one can merge stderr to stdout (most popular) with ‘2>&1’
  - **command** *> file 2>&1*

Pipes

- Using a pipe operator ‘|’ commands can be linked together. The pipe will link the standard output from one command to the standard input of another.

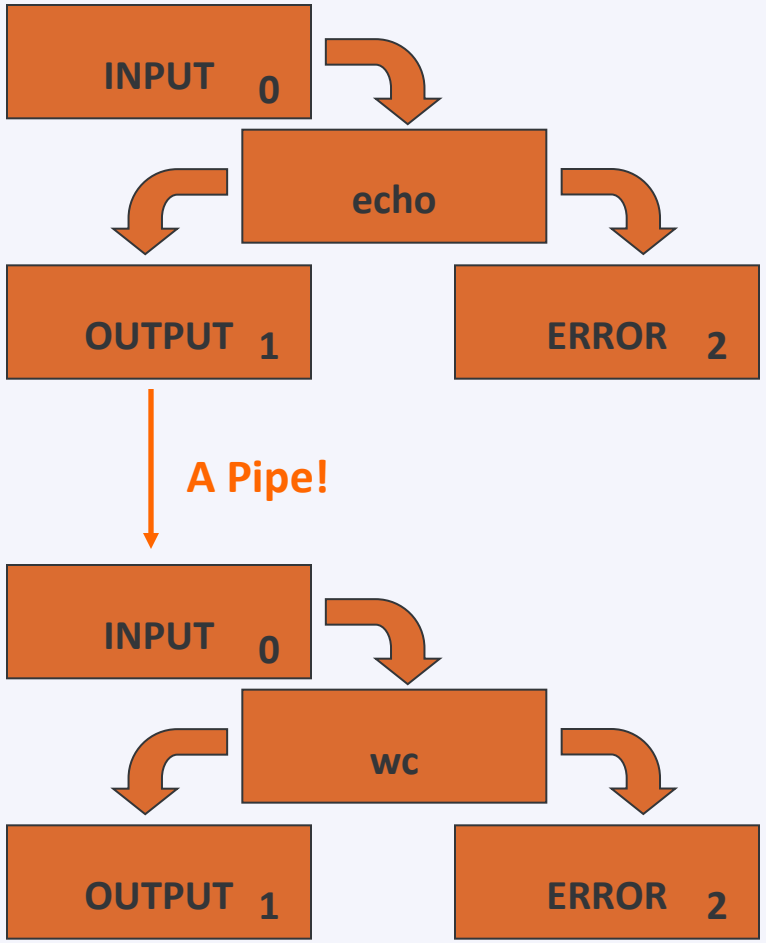
- Very helpful for searching files
- e.g. when we want to list the files, but only the ones that contain test in their name:

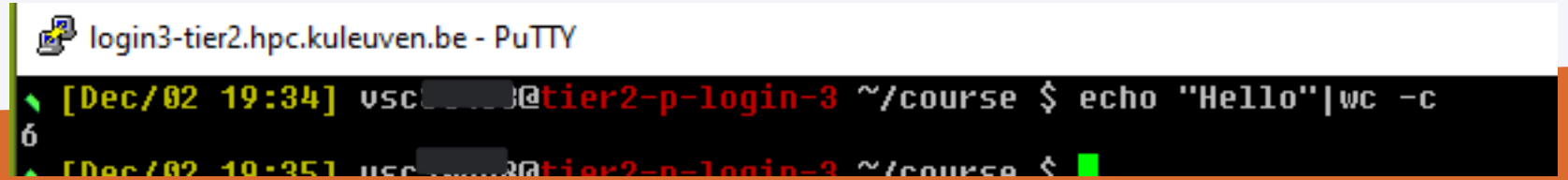
```
ls -la|grep test
```

Pipes

- Lots of Little Tools

```
echo “Hello” | \
  wc -c
```





wc flags

- -c, --bytes
  - print the byte counts
- -m, --chars
  - print the character counts
- -l, --lines
  - print the newline counts
- -L, --max-line-length
  - print the length of the longest line
- -w, --words
  - print the word counts
- Example
  - `wc file1`

Wc example

login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ cat n1.txt
a 1
b 2
c 3
d 4
e 5
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ wc n1.txt
 5 10 20 n1.txt
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ wc -c n1.txt
20 n1.txt
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ wc -m n1.txt
20 n1.txt
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ wc -l n1.txt
5 n1.txt
[Dec/02 19:39] vsc@tier2-p-login-3 ~/course/tools $ wc -w n1.txt
10 n1.txt
```

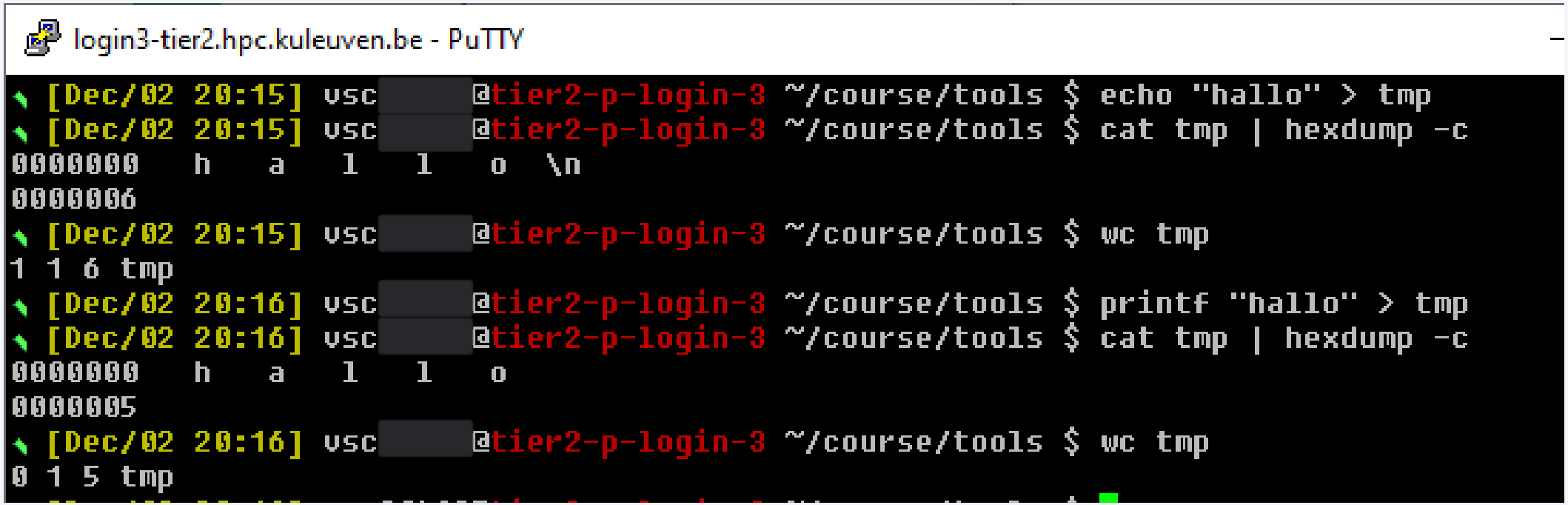
login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 19:40] vsc@tier2-p-login-3 ~/course/tools $ echo "Hallo" > hallo1.txt
[Dec/02 19:40] vsc@tier2-p-login-3 ~/course/tools $ printf "Hallo" > hallo2.txt
[Dec/02 19:41] vsc@tier2-p-login-3 ~/course/tools $ cat hallo1.txt
Hallo
[Dec/02 19:41] vsc@tier2-p-login-3 ~/course/tools $ cat hallo2.txt
Hallo[Dec/02 19:41] vsc@tier2-p-login-3 ~/course/tools $
[Dec/02 19:41] vsc@tier2-p-login-3 ~/course/tools $ wc hallo1.txt
1 1 6 hallo1.txt
[Dec/02 19:41] vsc@tier2-p-login-3 ~/course/tools $ wc hallo2.txt
0 1 5 hallo2.txt
```


VLAAMS SUPERCOMPUTER CENTRUM

Wc example

- `hexdump` - displays file contents in ascii, decimal, hexadecimal, or octal
  - -c: One-byte character display. Display the input offset in hexadecimal, followed by sixteen space-separated, three-column, space-filled characters of input data per line.

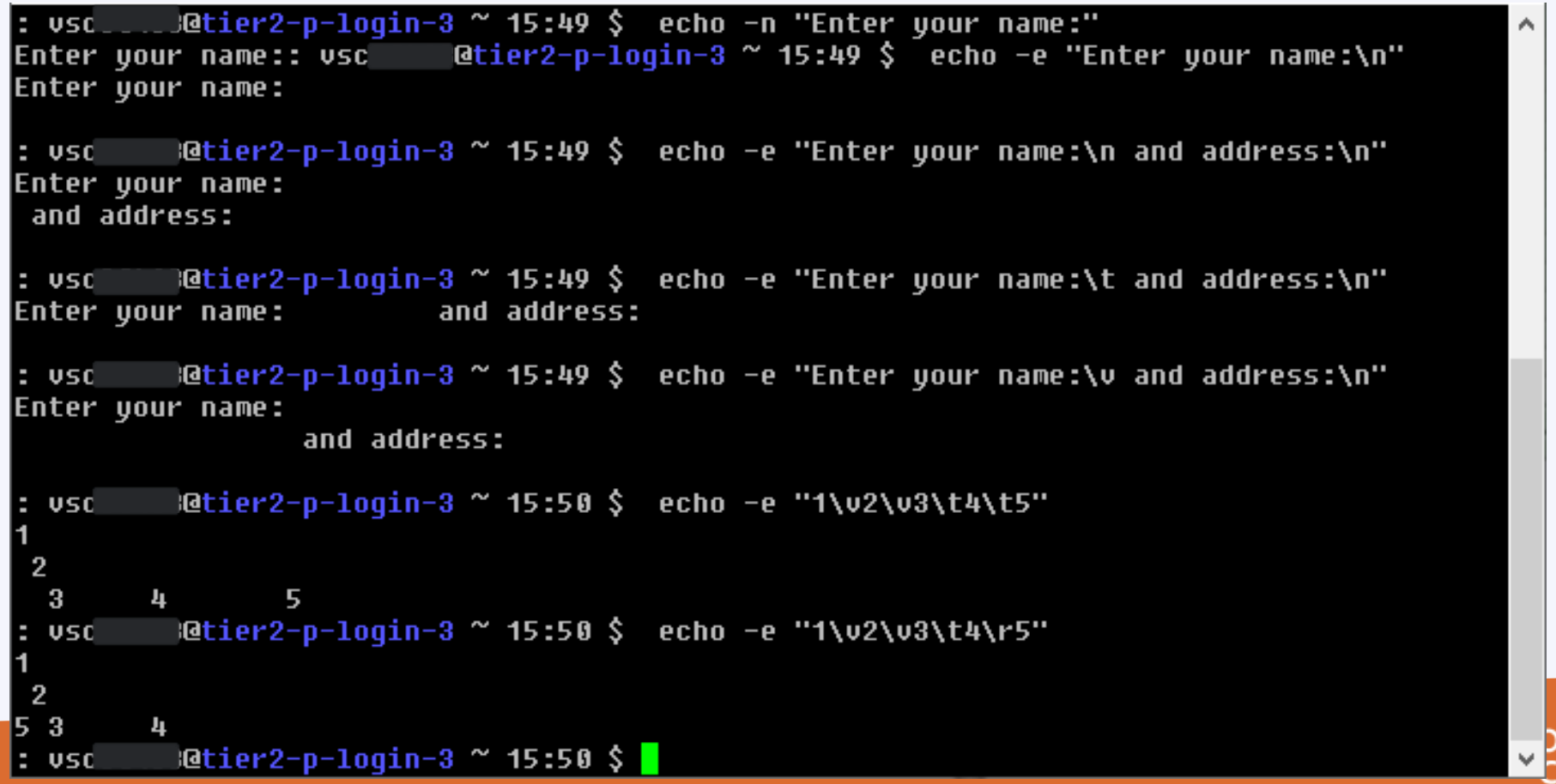


`\n` is the last character in the first line

Printing/output

Basic use: `echo` (remember quotes!)

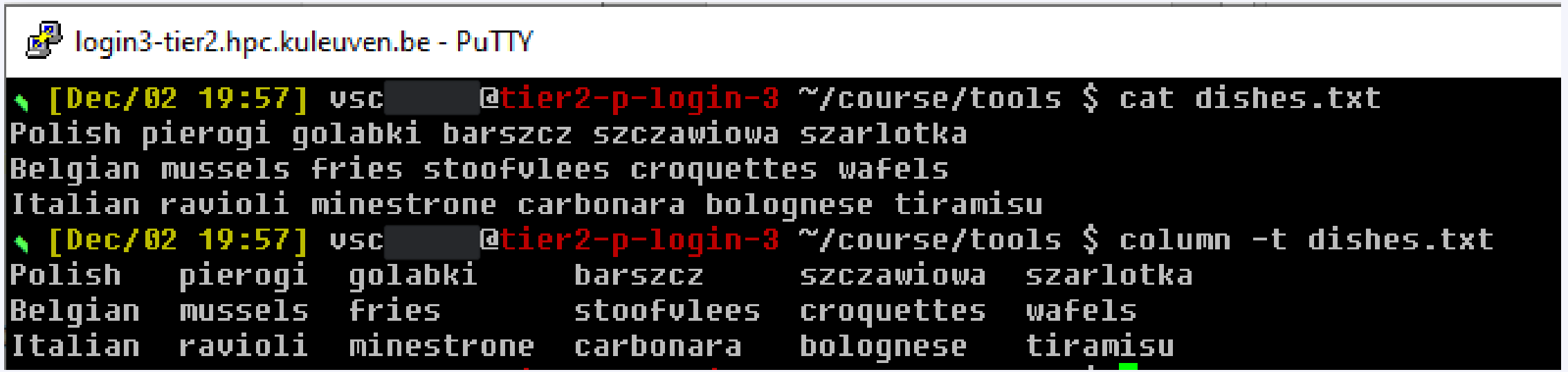
- `$ echo -n` -> do not output the trailing newline
- `$ echo -e` -> enable interpretation of backslash escapes
  - \n new line
  - \r carriage return
  - \t horizontal tab
  - \v vertical tab



Columnate lists – column flags

- -c, --columns width
  - Output is formatted to a width specified as number of characters.
- -t, --table
  - Determine the number of columns the input contains and create a table. Columns are delimited with whitespace, by default, or with the characters supplied using the separator.
- -s, --separator separators
  - Specify possible table delimiters (default is whitespace).
- -o, --output-separator separators
  - Specify table output delimiter (default is two whitespaces).
- -x, --fillrows
  - Fill columns before filling rows.
  - Example
  - `column file1.txt`

Column example



Comparing files and directories

- `$ diff file1 file2`
  Reports the differences between 2 files, or nothing if the files are identical.
- `$ diff -r dir1/ dir2/`
  Reports all the differences between files with the same name in the 2 directories.
- These differences can be saved in a file using the redirection, and then later re-applied.
- https://linuxacademy.com/blog/linux/introduction-using-diff-and-patch/

Defining local variables

- As in any other programming language, variables can be defined and used in shell scripts.
- Unlike other programming languages, variables in Shell Scripts are not typed.
- Examples :

```
a=1234  # a is NOT an integer, a string instead
b=$a+1 # will not perform arithmetic but be the string '1234+1'
b=`expr $a + 1 `  will perform arithmetic so b is 1235 now.
```

Note : +,-,/,*,**, % operators are available.

```
b=abcde # b is string
b='abcde' # same as above but much safer.
b=abc  def # will not work unless 'quoted'
b='abc def'  # i.e. this will work.
```

IMPORTANT: DO NOT LEAVE SPACES AROUND THE =

Some commands

- `cut` – cuts columns of text from a file
- `echo` – displays a line of text
- `paste` – will paste columns of text into a file
- `join` - operates on exactly two files, but pastes together only those lines with a common tagged field
- `sort` – sorts a file of lines alphabetically
- `comm` – compare 2 sorted files
- `tr` – translates between characters (e.g. tr a-z A-Z)
- `sleep` - Delay for a specified amount of time
- `uniq` - Remove duplicate lines from a sorted file

Paste flags

- -d
  - Delimiter
- -s
  - paste one file at a time instead of in parallel
- Example
  - `paste file1 file2`

Paste example

login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 17:54] vsc      @tier2-p-login-3 ~/course/tools $ cat ALPH.txt
A 1
B 2
C 3
D 4
E 5
F 6
G 7
H 8
I 9
J 10

[Dec/02 17:54] vsc      @tier2-p-login-3 ~/course/tools $ cat alph.txt
a I
b II
c III
d IV
e V
f VI
g VII
h VIII
i IX
j X

[Dec/02 17:54] vsc      @tier2-p-login-3 ~/course/tools $ paste ALPH.txt alph.txt
A 1     a I
B 2     b II
C 3     c III
D 4     d IV
E 5     e V
F 6     f VI
G 7     g VII
H 8     h VIII
I 9     i IX
J 10    j X
```

Paste example

login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 17:55] vsc     @tier2-p-login-3 ~/course/tools $ cat ALPH.txt
A 1
B 2
C 3
D 4
E 5
F 6
G 7
H 8
I 9
J 10

[Dec/02 17:55] vsc     @tier2-p-login-3 ~/course/tools $ cat alph-short.txt
a I
b II
c III
d IV
e V
f
g
h

[Dec/02 17:55] vsc     @tier2-p-login-3 ~/course/tools $ paste ALPH.txt alph-short.txt
A 1     a I
B 2     b II
C 3     c III
D 4     d IV
E 5     e V
F 6     f
G 7     g
H 8     h
I 9
J 10
```

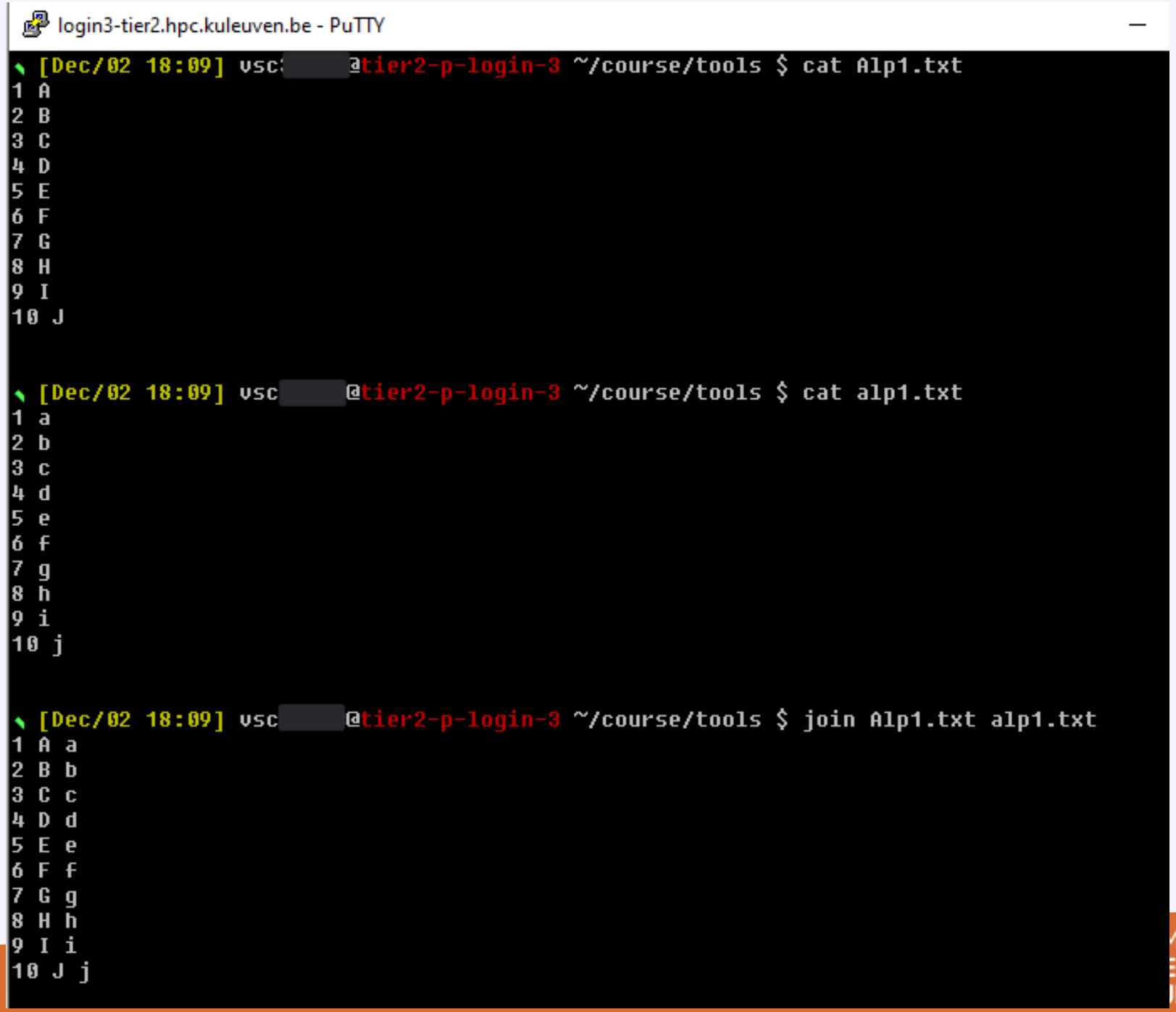
Join flags

- -i
  - Ignore case
- -e EMPTY
  - replace missing input fields with EMPTY
- -t CHAR
  - use CHAR as input and output field separator
- -1 FIELD
  - join on this FIELD of file 1
- -2 FIELD
  - join on this FIELD of file 2

Join flags

- -a FILENUM
  - Also, print unpairable lines from file FILENUM, where FILENUM is 1 or 2, corresponding to FILE1 or FILE2.
- -v
  - If you only want to print unpairable lines *i.e* suppress the paired lines in the output
- -j
  - In case, we have the position of common field same in both the files(other than first) then we can simply replace the part -1[field] -2[field] in the command with -j[field]
- Example
  - `join file1 file2`

Join example



Join example

login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 18:10] vsc      @tier2-p-login-3 ~/course/tools $ cat Alp2.txt
A  1
B  2
C  3
D  4
E  5
F  6
G  7
H  8
I  9
J 10

[Dec/02 18:10] vsc      @tier2-p-login-3 ~/course/tools $ cat alp1.txt
1 a
2 b
3 c
4 d
5 e
6 f
7 g
8 h
9 i
10 j

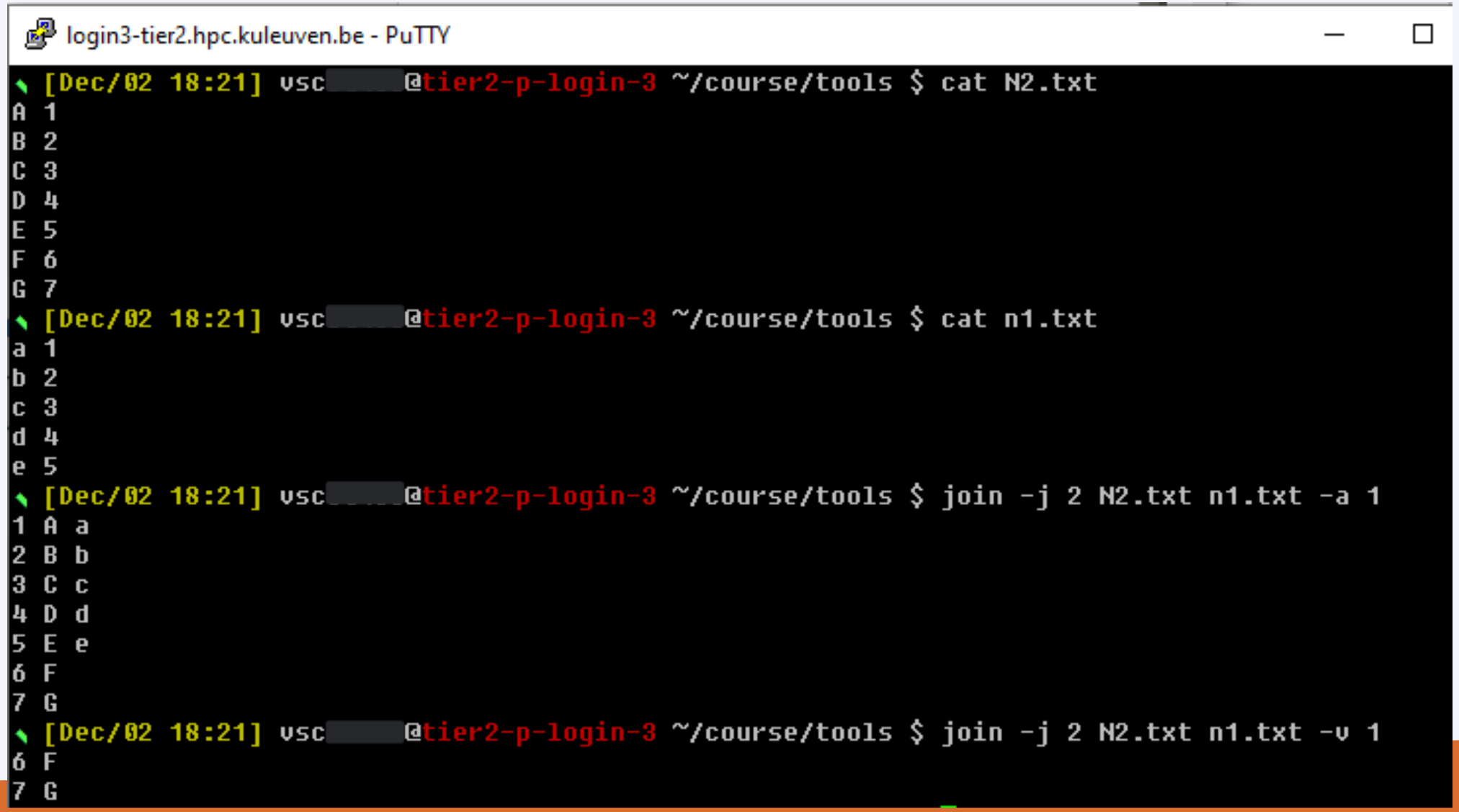

[Dec/02 18:10] vsc      @tier2-p-login-3 ~/course/tools $ join -1 2 -2 1 Alp2.txt  alp1.txt
1 A a
2 B b
3 C c
4 D d
5 E e
6 F f
7 G g
8 H h
9 I i
10 J j
```

Join example

login3-tier2.hpc.kuleuven.be - PuTTY

```
[Dec/02 18:13] vsc[redacted]@tier2-p-login-3 ~/course/tools $ cat N1.txt
A 1
B 2
C 3
D 4
E 5
[Dec/02 18:13] vsc[redacted]@tier2-p-login-3 ~/course/tools $ cat n1.txt
a 1
b 2
c 3
d 4
e 5
[Dec/02 18:13] vsc[redacted]@tier2-p-login-3 ~/course/tools $ join -j 2 N1.txt n1.txt
1 A a
2 B b
3 C c
4 D d
5 E e
[Dec/02 18:13] vsc[redacted]@tier2-p-login-3 ~/course/tools $ join -i N1.txt n1.txt
A 1 1
B 2 2
C 3 3
D 4 4
E 5 5
```

Join example



VLAAMS SUPERCOMPUTER CENTRUM

Sort flags

- -r
  - reverse
- -f
  - ignore case
- -k`n`
  - sort according to column `n`
- `-n`
  - compare according to string numerical value
- -V
  - natural sort of (version) numbers within text (for files like file-1.txt, …, file-100.txt)
- Example
  - `sort -r -k3 < myFile`

Sort example

```
: vsc      @hpc-p-login-2 ~ 05:42 $ cat TestSort
J is the tenth letter of the alphabet.
B is the second letter of the alphabet.
C is the third letter of the alphabet.
G is the seventh letter of the alphabet.
A is the first letter of the alphabet.
D is the forth letter of the alphabet.
F is the sixth letter of the alphabet.
H is the eighth letter of the alphabet.
E is the fith letter of the alphabet.
I is the nineth letter of the alphabet.
: vsc      @hpc-p-login-2 ~ 05:42 $ sort TestSort
A is the first letter of the alphabet.
B is the second letter of the alphabet.
C is the third letter of the alphabet.
D is the forth letter of the alphabet.
E is the fith letter of the alphabet.
F is the sixth letter of the alphabet.
G is the seventh letter of the alphabet.
H is the eighth letter of the alphabet.
I is the nineth letter of the alphabet.
J is the tenth letter of the alphabet.
: vsc      @hpc-p-login-2 ~ 05:42 $
```

Uniq example

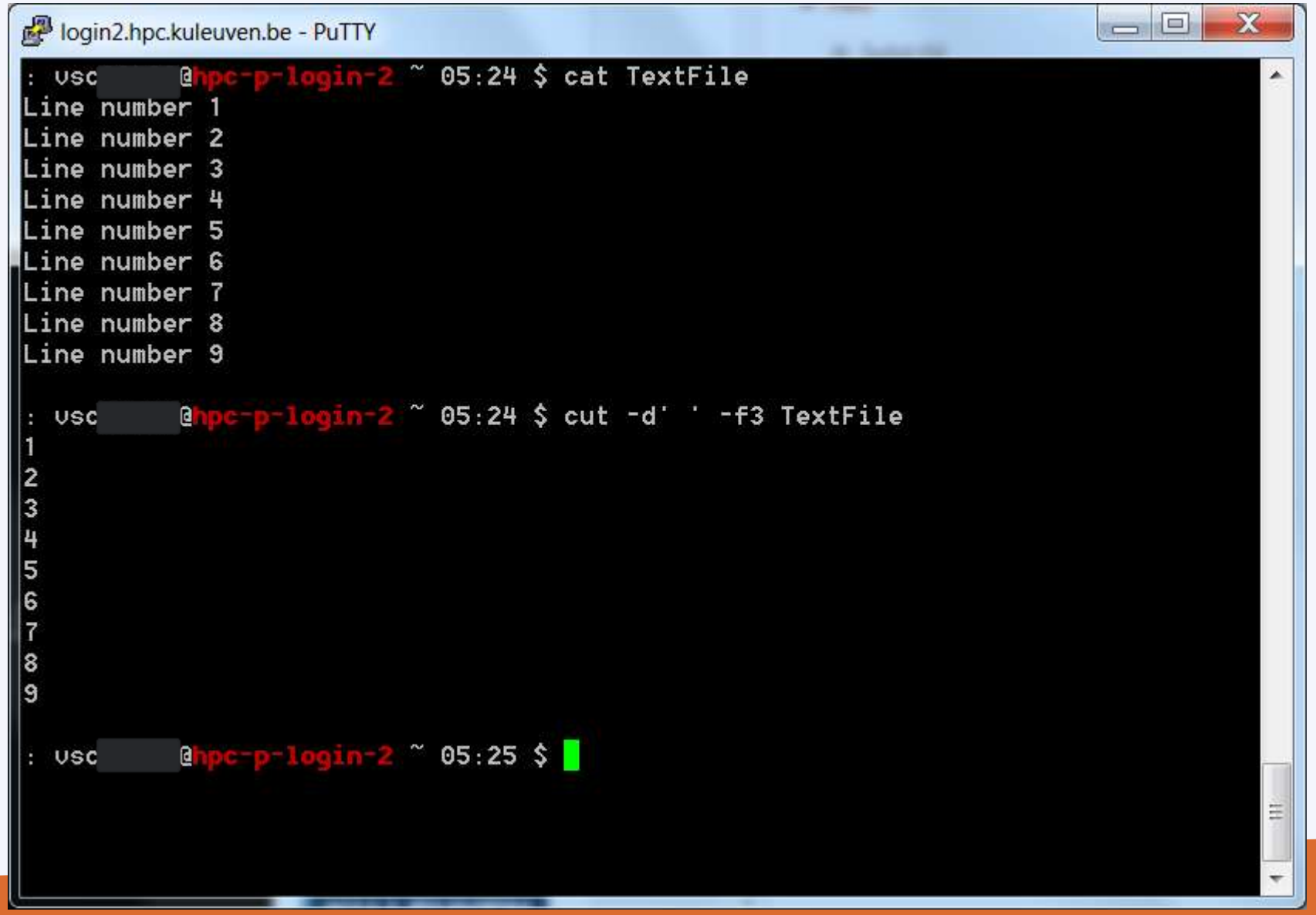
```
: vsc[redacted]@hpc-p-login-2 ~ 05:46 $ cat TestUni
B is the second letter of the alphabet.
C is the third letter of the alphabet.
C is the third letter of the alphabet.
D is the fourth letter of the alphabet.
A is the first letter of the alphabet.
A is the first letter of the alphabet.
D is the fourth letter of the alphabet.
B is the second letter of the alphabet.
: vsc[redacted]@hpc-p-login-2 ~ 05:46 $ uniq TestUni
B is the second letter of the alphabet.
C is the third letter of the alphabet.
D is the fourth letter of the alphabet.
A is the first letter of the alphabet.
D is the fourth letter of the alphabet.
B is the second letter of the alphabet.
: vsc[redacted]@hpc-p-login-2 ~ 05:46 $ sort TestUni
A is the first letter of the alphabet.
A is the first letter of the alphabet.
B is the second letter of the alphabet.
B is the second letter of the alphabet.
C is the third letter of the alphabet.
C is the third letter of the alphabet.
D is the fourth letter of the alphabet.
D is the fourth letter of the alphabet.
: vsc[redacted]@hpc-p-login-2 ~ 05:46 $ sort TestUni | uniq
A is the first letter of the alphabet.
B is the second letter of the alphabet.
C is the third letter of the alphabet.
D is the fourth letter of the alphabet.
: vsc[redacted]@hpc-p-login-2 ~ 05:46 $
```

`sort -u TestUni`
will do the same

Cut flags

- -d
  - Delimiter
- -f
  - Field number
- Example
  - `cut -d' '  -f3 myFile`

Example Of Cut



comm flags

- Without any options, it prints output in three columns - lines unique to file1, line unique to file2 and lines common to both files
- -1
  - suppress column 1 (lines unique to FILE1)
- -2
  - suppress column 2 (lines unique to FILE2)
- -3
  - suppress column 3 (lines that appear in both files)

comm flags

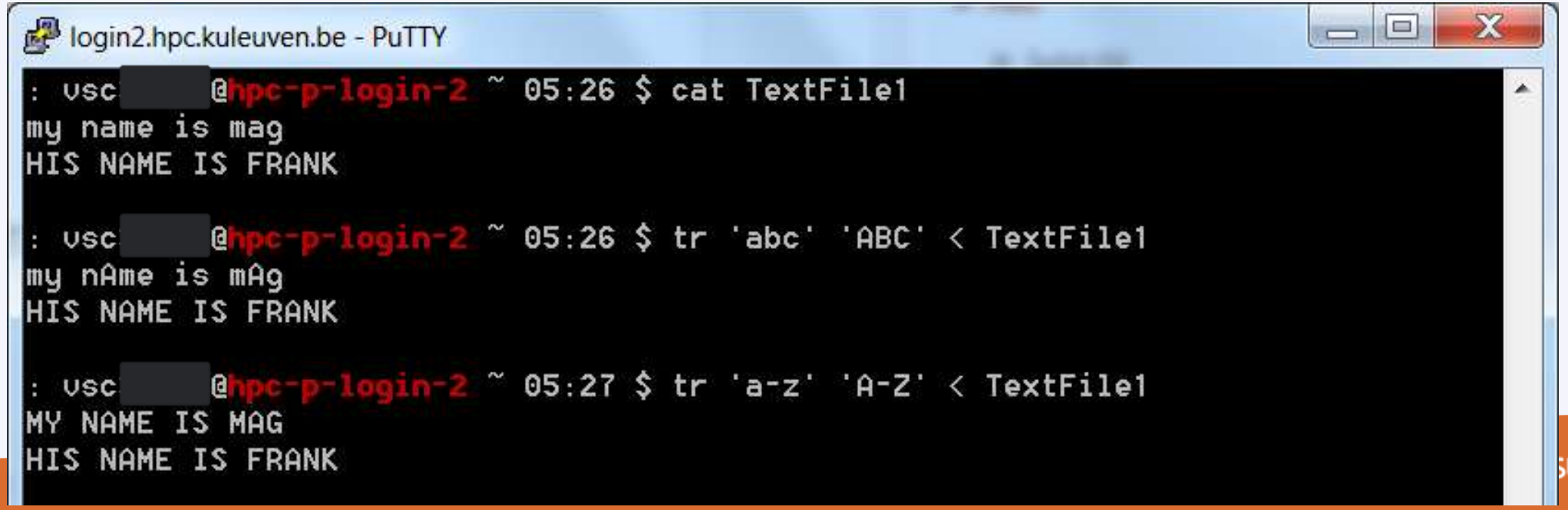
- `--check-order`
  - check that the input is correctly sorted, even if all input lines are pairable
- `--nocheck-order`
  - do not check that the input is correctly sorted
- `--output-delimiter=STR`
  - separate columns with STR
- Example
  - `comm file1 file2`

Comm example

```
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ cat N1.txt
A 1
B 2
C 3
D 4
E 5
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ cat N1-further.txt
A 1
B 2
C 3
F 6
G 7
H 8
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ comm N1.txt N1-further.txt
                A 1
                B 2
                C 3
D 4
E 5
        F 6
        G 7
        H 8
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ comm -1 N1.txt N1-further.txt
        A 1
        B 2
        C 3
F 6
G 7
H 8
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ comm -2 N1.txt N1-further.txt
        A 1
        B 2
        C 3
D 4
E 5
[Dec/02 20:08] vsc[redacted]tier2-p-login-3 ~/course/tools $ comm -12 N1.txt N1-further.txt
A 1
B 2
C 3
[Dec/02 20:09] vsc[redacted]tier2-p-login-3 ~/course/tools $ comm -3 N1.txt N1-further.txt
D 4
E 5
        F 6
        G 7
        H 8
```

The tr Command

- Translate
- Change on a one-to-one basis characters from one thing to another
- Usage: `tr ‘Set1’ ‘Set2’`
  - Example: `tr ‘abc’ ‘ABC’ < myFile`
  - `tr '[:lower:]' '[:upper:]' < myFile`



Lower/Upper case letters

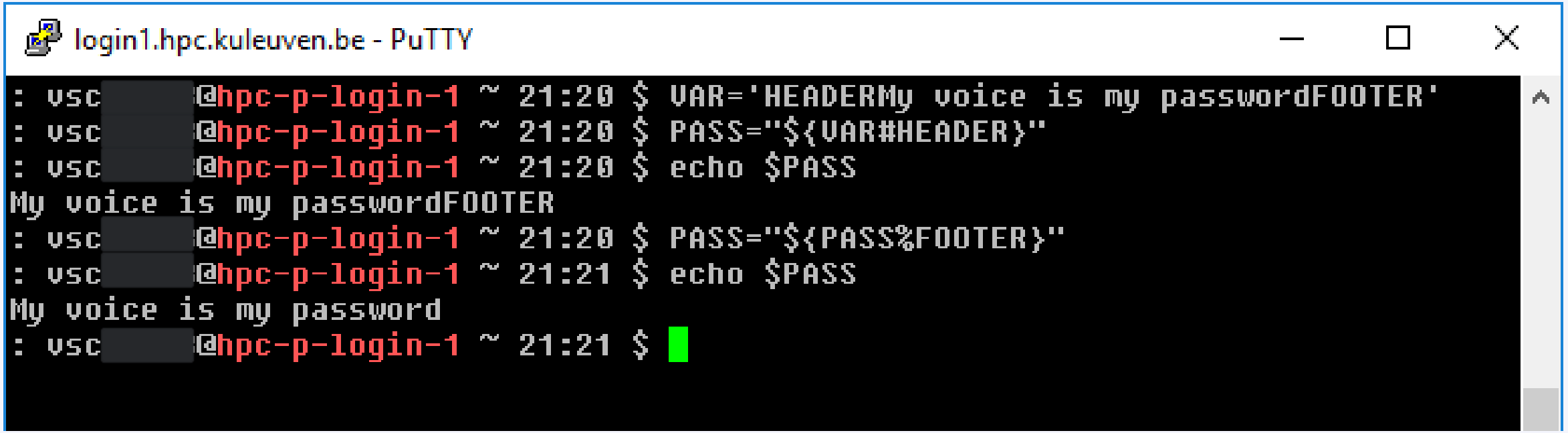
- Capitalize:
  - ^^ - all the letters
  - ^ - only the first letter

- Lowercase:
  - ,, - all the letters
  - , - only the first letter

```
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ string='hello world!'
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ echo ${string}
hello world!
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ echo ${string^}
Hello world!
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ echo ${string^^}
HELLO WORLD!
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ STRING='HELLO WORLD!'
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ echo ${STRING}
HELLO WORLD!
: vsc[redacted]@hpc-p-login-1 ~ 10:50 $ echo ${STRING,}
hELLO WORLD!
: vsc[redacted]@hpc-p-login-1 ~ 10:51 $ echo ${STRING,,}
hello world!
```

String manipulation

- The # means ‘match and remove the following pattern from the start of the string’
- The % means ‘match and remove the following pattern from the end of the string



Globbing: use wildcard

Wildcard	Function
*	Matches 0 or more characters
?	Matches 1 character
[abc]	Matches one of the characters listed
[a-c]	Matches one character in the range
[!abc]	Matches any character not listed
[!a-c]	Matches any character not listed in the range
{tacos,nachos}	Matches one word in the list

grep – Global / Regular Expressions / Pattern

- Searches the internals of files and tries to match patterns
- Used to see if a file contains data you are looking for
- Will print out every line that contains a match for that pattern
- Usage: grep [OPTIONS] pattern [FILE]

Useful tools

- `grep`
  - Pattern searching
  - Example: `grep parameters pattern filename`
  - `-v` (invert the sense of matching, to select non-matching lines)
  - `-i` (ignore case)
  - `-e/-E` (Look for expression/extended expression)

  - ```
    $ grep ";E;" $PBSLOGSDIR/$file | grep -E
    "gpu1\/|gpu2\/|gpu3" > $file
    ```
    ```

Useful tools

```
: vsc      @hpc-p-login-2 ~ 05:38 $ cat TestAlphabet
A is the first letter of the alphabet.
B is the second letter of the alphabet.
C is the third letter of the alphabet.
D is the forth letter of the alphabet.
E is the fith letter of the alphabet.
F is the sixth letter of the alphabet.
G is the seventh letter of the alphabet.
H is the eighth letter of the alphabet.
I is the nineth letter of the alphabet.
J is the tenth letter of the alphabet.
T is the twentieth letter of the alphabet.
U is the twenty-first letter of the alphabet.
V is the twenty-second letter of the alphabet.
W is the twenty-third letter of the alphabet.
X is the twenty-fourth letter of the alphabet.
Y is the twenty-fifth letter of the alphabet.
Z is the twenty-sixth letter of the alphabet.
: vsc      @hpc-p-login-2 ~ 05:38 $ grep first TestAlphabet
A is the first letter of the alphabet.
U is the twenty-first letter of the alphabet.
: vsc      @hpc-p-login-2 ~ 05:38 $
```

Common Flags

- `-i`
  - Case insensitive (upper and lower cases are treated the same)
- `-n`
  - Print out the line numbers
- `-r`
  - Recursively traverse the directory
- `-v`
  - Invert the results (show all non-matching lines)
- `-l`
  - Shows only the name of matching file, not the matches (useful in combination with find)
- `alias grep='grep --color=auto'`
  - Possible alias in .bashrc to highlight matches in the output, making it easy to spot them.

Easiest grep Usage

- The easiest way to use grep is also the most common way to use grep
  - Search files for occurrences of a string (word)
- The pattern you search for can simply be a word

Regular Expressions

- grep can be used to find words that match a certain pattern, not just a given word
- The language of regular expressions is used to describe these patterns
- This includes wildcards, repetitions, and complex patterns

How grep Views Regular Expressions

- Unfortunately, grep's regular expressions are completely different than the shell wildcards
- Some of the symbols are the same, but they are used in different ways
- Always use quotes (') so that the wildcards are interpreted by grep and not the shell

Notation

- ^
  - Beginning of the line – left rooted
- $
  - End of the line – right rooted
- .
  - Any single character
- [xy]
  - Any character in the set
- [^a-z]
  - Any character not in the set
- B*
  - Zero or more occurrences of B

Examples

- .*
  - Zero or more of any character
  - Will match any pattern
- ^ab*
  - Any line that starts with a and has zero or more b’s immediately following
    - ab
    - abbbb
    - abb

Examples

- [0-9]
  - Any number
    - 1002
    - 0909
- bye$
  - The pattern “bye” located at the end of the line
    - Hello and goodbye

One More Slide Of Examples…

- [^g]$
  - Match any line that does not end in g
- [:alpha:]*
  - Any word that contains zero or more alphabetic characters

Inverting The Answers

- grep –v ‘this’ testFile
  - Will find all lines that do not contain the word this
  - Works exactly the same with regular expressions
- grep –v ‘[^g$]’ testFile
  - Finds all lines that end in g

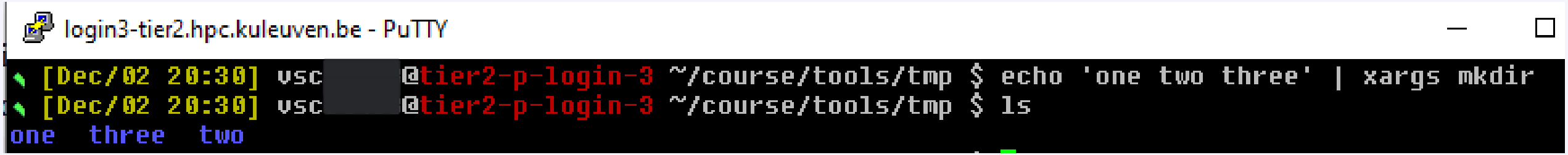
Your own command?

- The `xargs` command in UNIX is a command line utility for building an execution pipeline from standard input. Whilst tools like `grep` can accept standard input as a parameter, many other tools cannot. Using `xargs` allows tools like `echo` and `rm` and `mkdir` to accept standard input as arguments.

- By default, `xargs` reads items from standard input as separated by blanks and executes a command once for each argument.

Your own command?

- In the following example standard input is piped to xargs and the mkdir command is run for each argument, creating three folders.



xargs+find

- The most common usage of `xargs` is to use it with the `find` command. This uses `find` to search for files or directories and then uses `xargs` to operate on the results. Typical examples of this are changing the ownership of files or moving files.

- `find` and `xargs` can be used together to operate on files that match certain attributes.

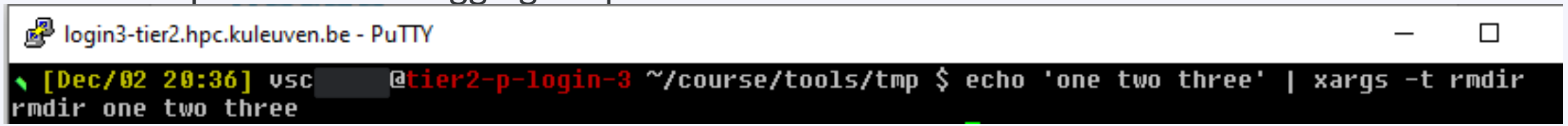
xargs+find

- In the following example files older than two weeks in the temp folder are found and then piped to the `xargs` command which runs the `rm` command on each file and removes them.

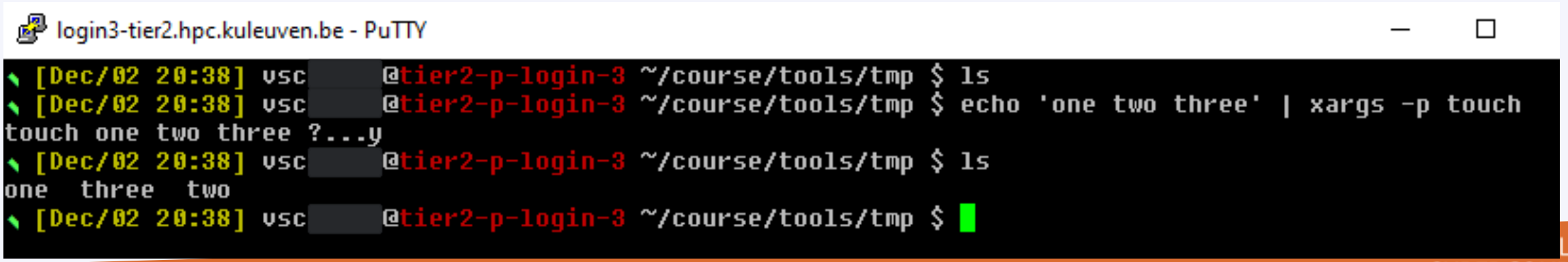
```
$ find /tmp -mtime +14 | xargs rm
```

xargs

- xargs can print commands that are executed
  - The -t option prints each command that will be executed to the terminal. This can be helpful when debugging scripts.



- xargs allows to view the command and prompt for execution
  - The -p command will print the command to be executed and prompt the user to run it. This can be useful for destructive operations where you really want to be sure on the command to be run.



Hands-on 1

1. Download the file https://github.com/hpcleuven/Linux-tools/blob/master/ex1.fa
2. and change all the capital letter A and C into lower case (save into a new file)
3. Modify ex1.fa file so that A is changed into B, C into D, etc. (save into a new file)
4. Create a file containing some information, e.g. "*Hello Mag, it is a nice sunny day.*" Write a conversion tool for Rot13 (https://en.wikipedia.org/wiki/ROT13). Apply it once to the file, check the output, apply again and compare the output with the original file.
5. Download the file https://github.com/hpcleuven/Linux-tools/blob/master/lp_hpcinfo and save into a new file only column 5
6. Remove duplicated lines from the file
7. Sort lp_hpcinfo according to the 5th column
8. Download the file https://github.com/hpcleuven/Linux-tools/blob/master/20170601 and print only lines that contain Exit_status=0
9. From 20170601 print only lines that do not contain ";E;"
10. From 20170601 print only lines that contain r3i0n11 or r3i0n12
11. From 20170601 print only lines that contain both ";E; and "Exit_status=0"

1. tr [A-C] [a-c] < ex1.fa > ex1a.fa
2. tr '[A-Z]' '[B-ZA]' <ex1.fa
3. tr A-Za-z N-ZA-Mn-za-m < r13inp.txt > r13out1.txt
   tr A-Za-z N-ZA-Mn-za-m < r13out1.txt > r13out2.txt
   diff rot13-input.txt rot13-output2.txt
5. cut -d' ' -f5 lp_hpcinfo > hpcinfo.log
6. sort hpcinfo.log |uniq > hpcinfo
   or: sort –u hpcinfo.log > hpcinfo
7. sort lp_hpcinfo -k 5
8. grep "Exit_status=0" 20170601
9. grep -v ";E;" 20170601
10. grep -E "r3i0n11|r3i0n12" 20170601
    or grep -E "r3i0n1[1-2]" 20170601
11. grep "Exit_status=0" 20170601 |grep ";E;"

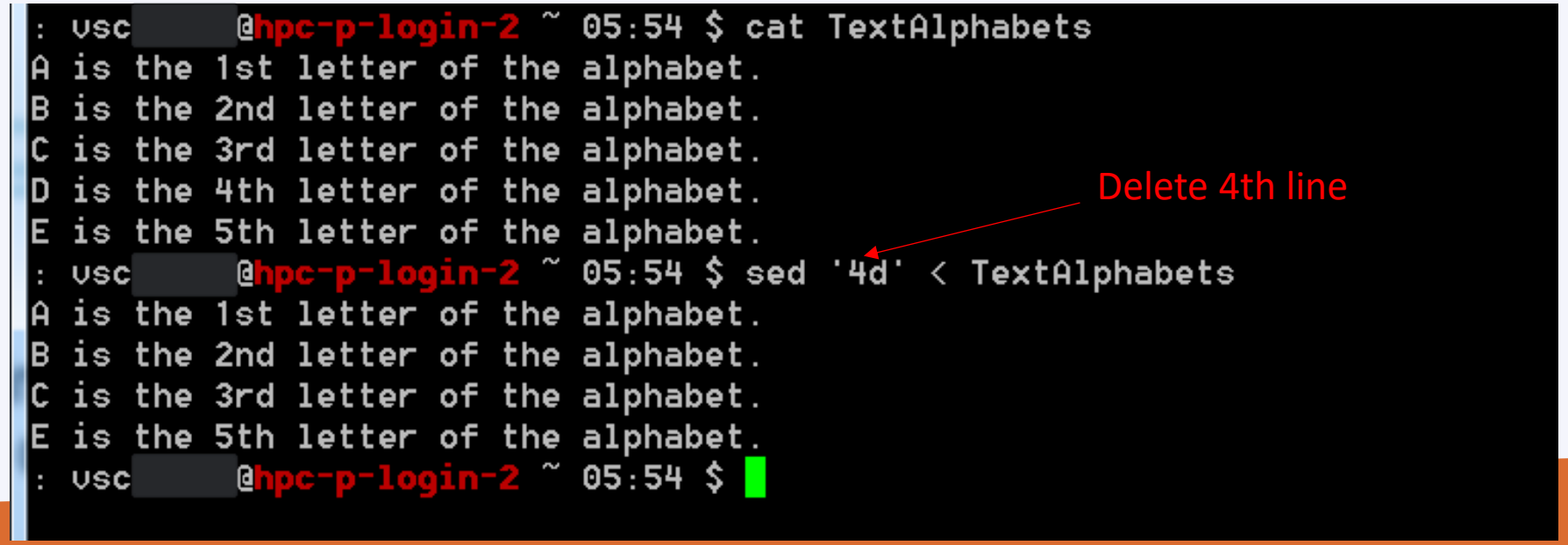
Useful tools

- `Sed` (Stream Editor)
  - Text editing
  - Example: `sed 's/XYZ/xyz/g' filename`

- `Awk` (Alfred Aho, Peter Weinberger, and Brian Kernighan)
  - Pattern scanning and processing
  - Example: `awk '{print $4, $7}' filename`

The ‘sed’ Command/Language

- Filter
  - Like grep, sort, or uniq, it takes input and performs some operation on it to filter the output
- Usage: sed ‘Address Command’
  - Address specifies where to make changes
  - Command specifies what change to make
  - Example:
    - sed ‘4d’ textFile



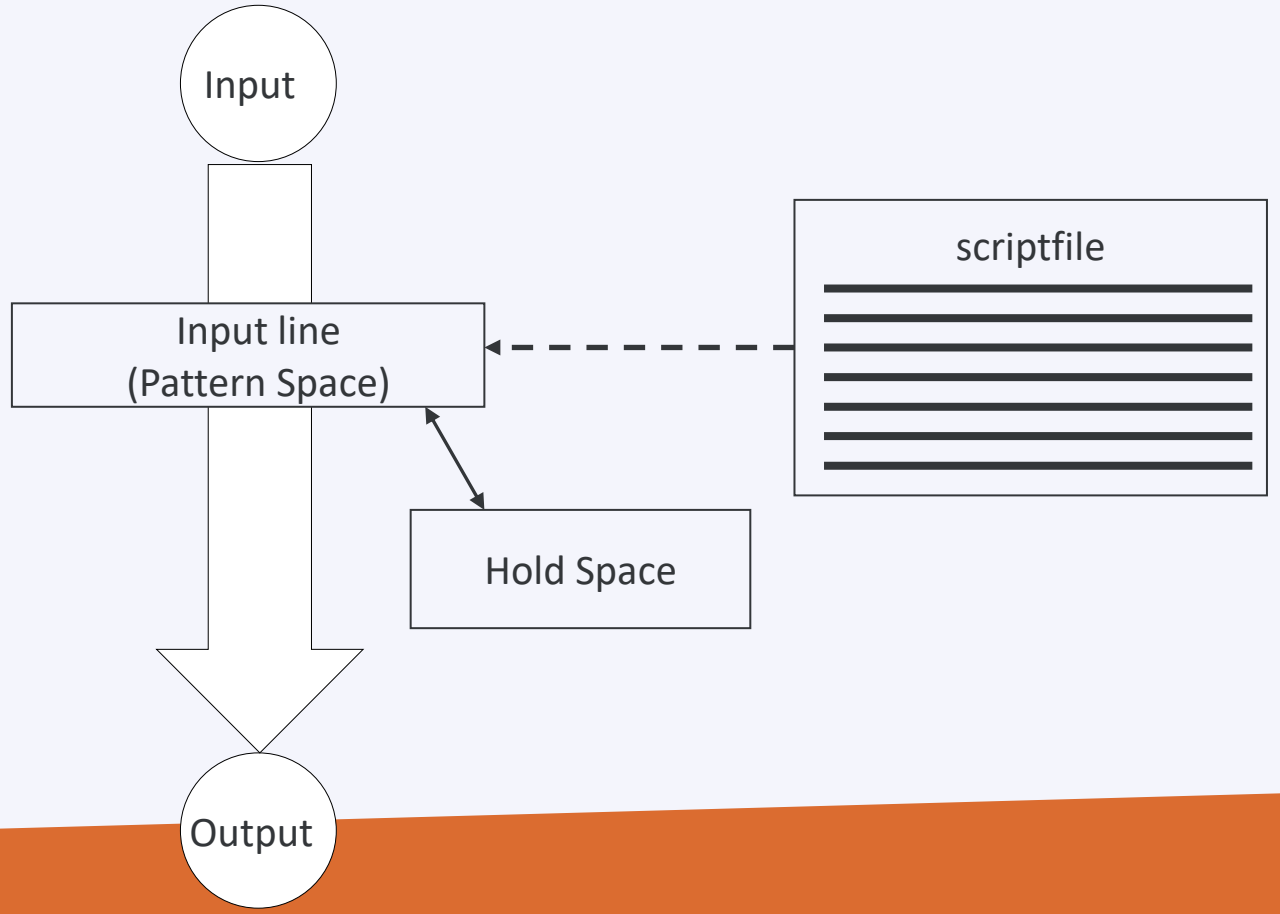
How Does sed Work?

- sed reads line of input
  - line of input is copied into a temporary buffer called pattern space
  - editing commands are applied
    - subsequent commands are applied to line in the pattern space, not the original input line
    - once finished, line is sent to output (unless –n option was used)
  - line is removed from pattern space
- sed reads next line of input, until end of file

Note: input file is unchanged

Pattern and Hold spaces

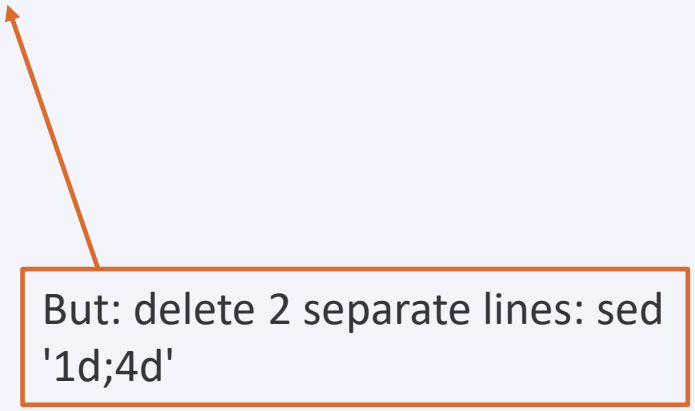
- Pattern space: Workspace or temporary buffer where a single line of input is held while the editing commands are applied
- Hold space: Secondary temporary buffer for temporary storage only



Address Specification

Addresses could be line numbers or regular expressions

- No address – each line
- One address – only that line
- Two comma separated addresses – All lines in between
- ! – All other lines



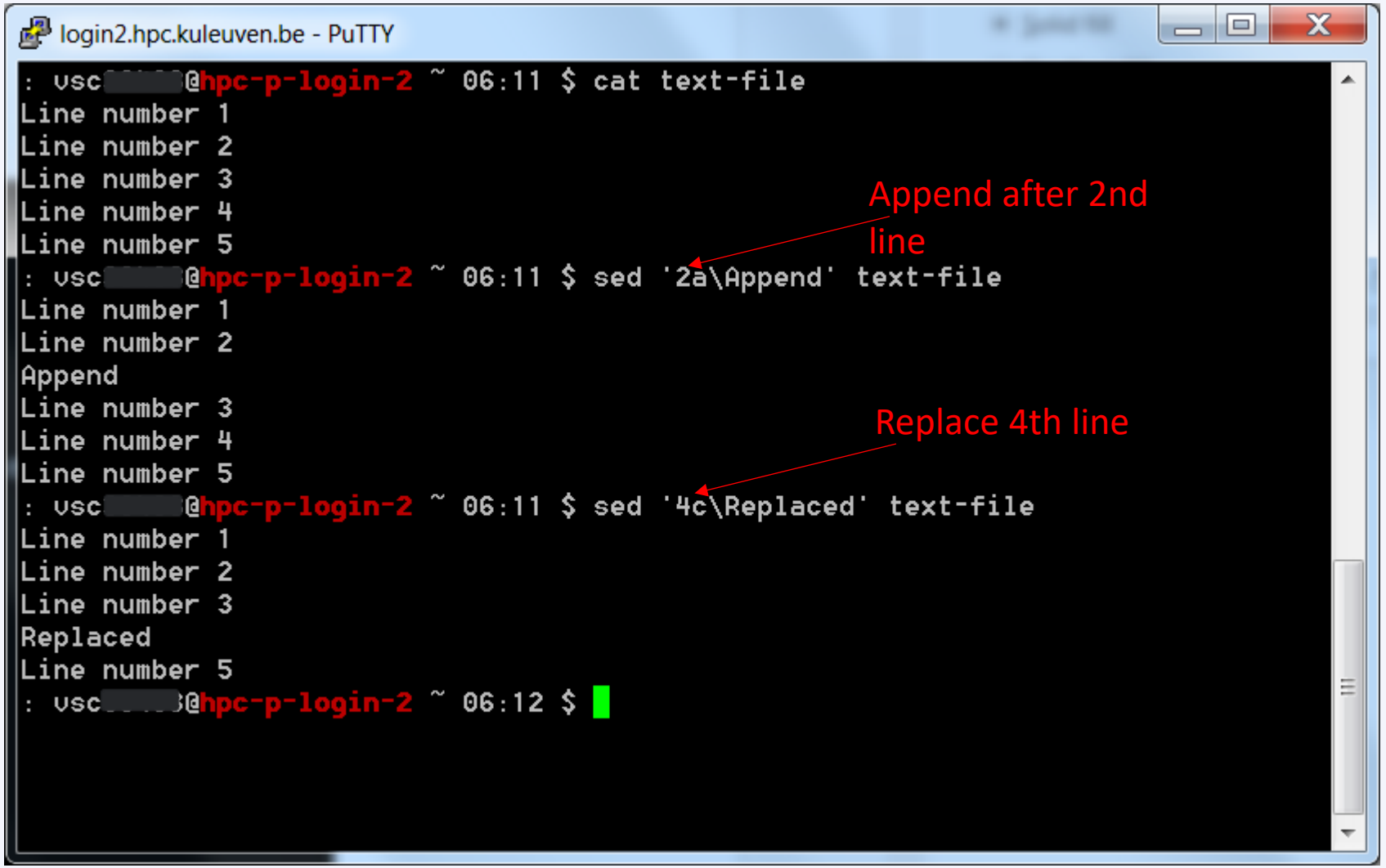
Some commands Available To sed

- a\
  - Append text
- c\
  - Replace text
- i\
  - Insert text before
- d
  - Delete lines
- s
  - Make substitutions

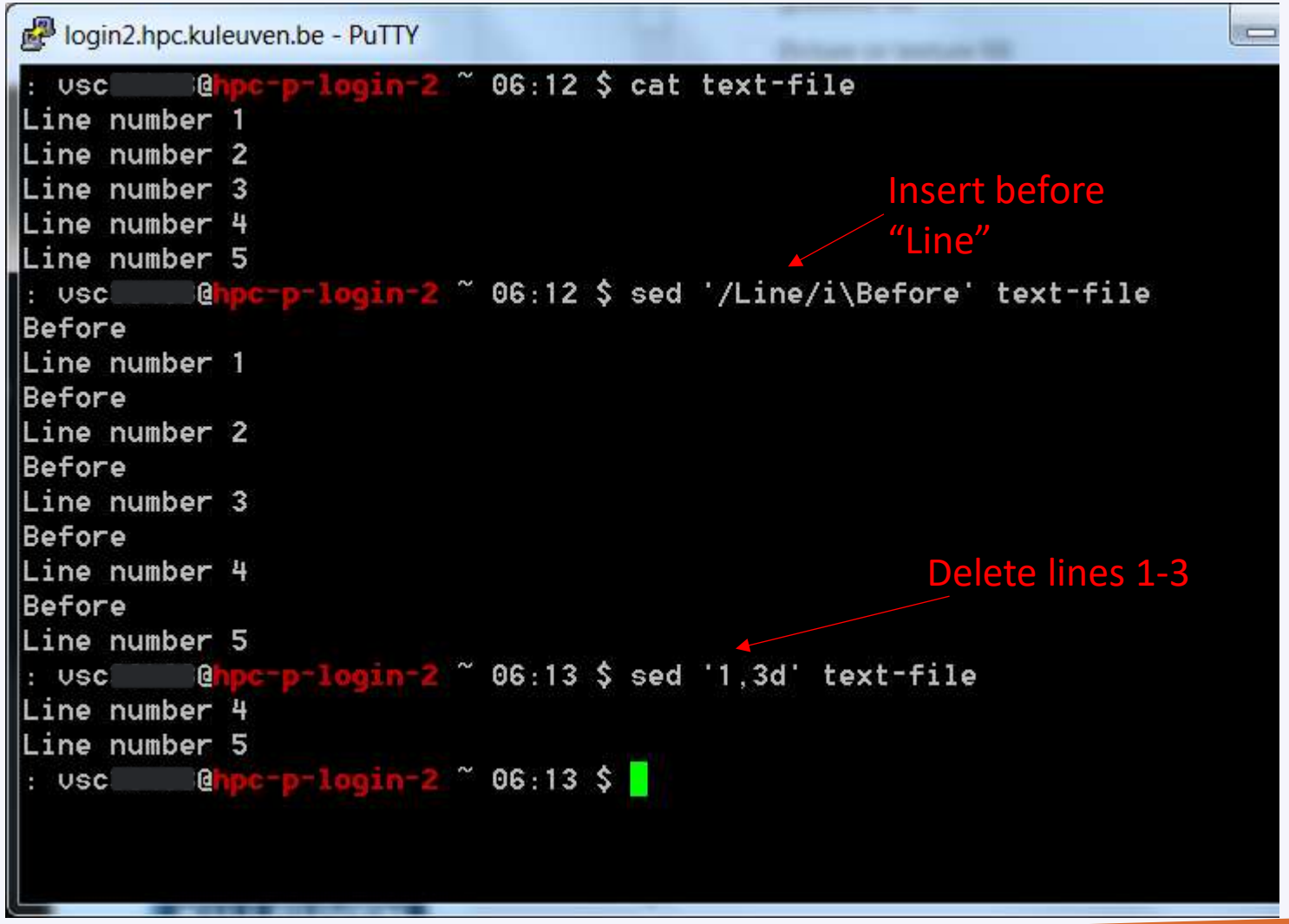
More sed commands

- **p** - print
- **r** - read
- **w** - write
- **y** - transform
- = - display line number
- N - append the next line to the current one
- **q** - quit

Examples


login2.hpc.kuleuven.be - PuTTY
: vsc[redacted]@hpc-p-login-2 ~ 06:11 $ cat text-file
Line number 1
Line number 2
Line number 3
Line number 4
Line number 5
Append after 2nd line
: vsc[redacted]@hpc-p-login-2 ~ 06:11 $ sed '2a\Append' text-file
Line number 1
Line number 2
Append
Line number 3
Line number 4
Line number 5
Replace 4th line
: vsc[redacted]@hpc-p-login-2 ~ 06:11 $ sed '4c\Replaced' text-file
Line number 1
Line number 2
Line number 3
Replaced
Line number 5
: vsc[redacted]@hpc-p-login-2 ~ 06:12 $

More Examples

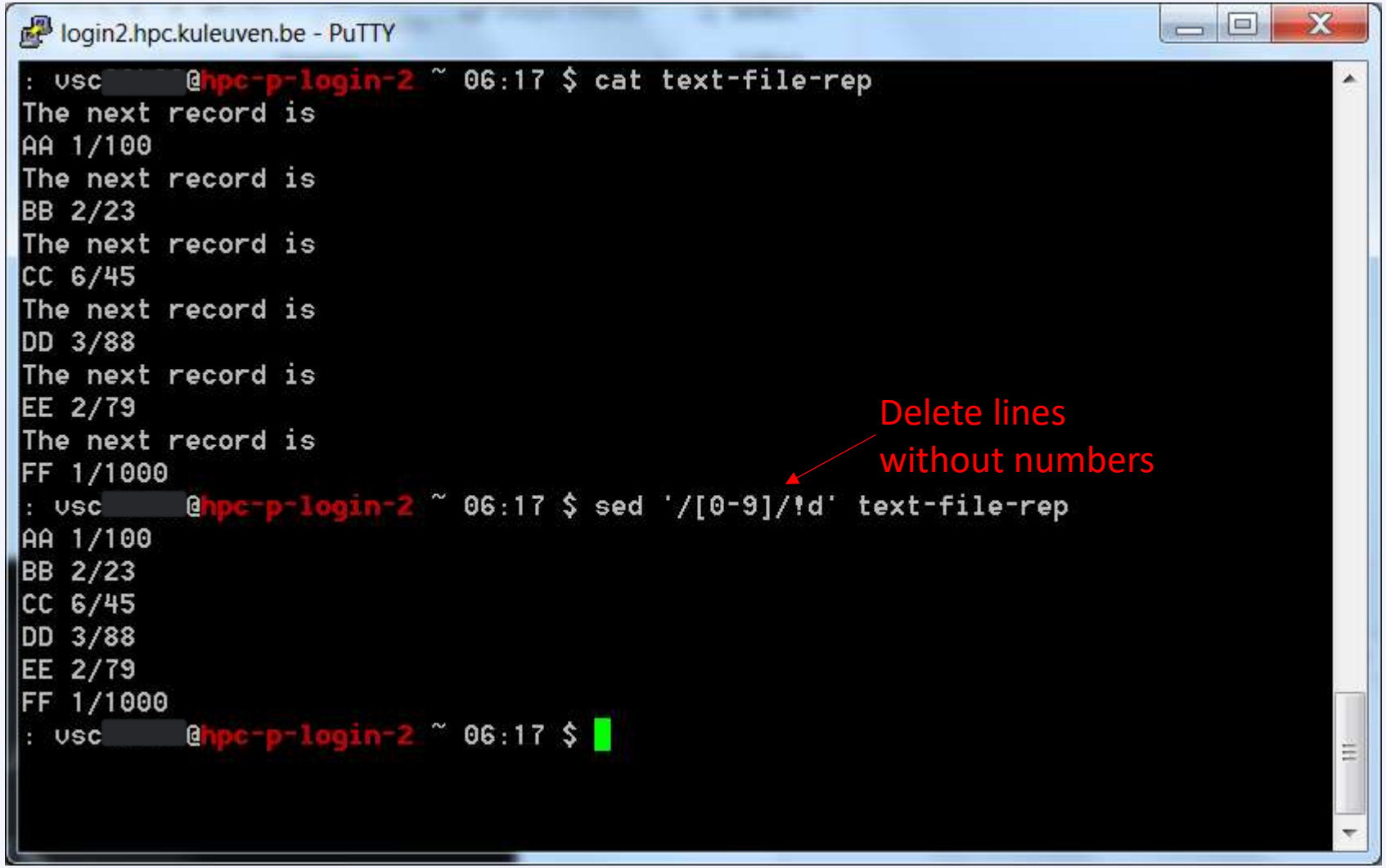


When Would You Want To Use sed?

- sed works on streams, so it is perfect to be placed in the middle of a pipe in order to change the output from one format to another

- Example:
  - If a program always prints out 4 lines of junk for every good line, sed can be used to weed out the junk

Example


login2.hpc.kuleuven.be - PuTTY
: vsc[redacted]@hpc-p-login-2 ~ 06:17 $ cat text-file-rep
The next record is
AA 1/100
The next record is
BB 2/23
The next record is
CC 6/45
The next record is
DD 3/88
The next record is
EE 2/79
The next record is
FF 1/1000
: vsc[redacted]@hpc-p-login-2 ~ 06:17 $ sed '/[0-9]/!d' text-file-rep
AA 1/100
BB 2/23
CC 6/45
DD 3/88
EE 2/79
FF 1/1000
: vsc[redacted]@hpc-p-login-2 ~ 06:17 $
Delete lines
without numbers

Sed Usage

- Edit files too large for interactive editing
- Edit any size files where editing sequence is too complicated to type in interactive mode
- Perform “multiple global” editing functions efficiently in one pass through the input
- Edit multiple files automatically
- Good tool for writing conversion programs

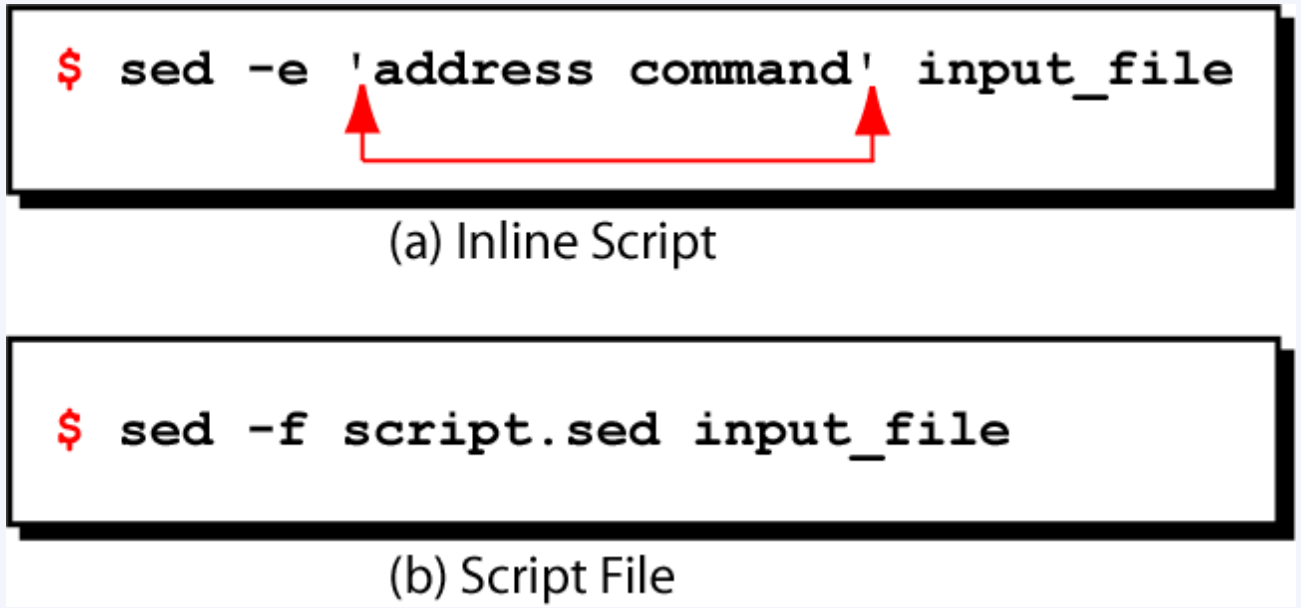
Conceptual overview

- A script is read which contains a list of editing commands
- Can be specified in a file or as an argument
- Before any editing is done, all editing commands are compiled into a form to be more efficient during the execution phase.
- All editing commands in a sed script are applied in order to each input line.
- If a command changes the input, subsequent command address will be applied to the current (modified) line in the pattern space, not the original input line.
- The original input file is unchanged (sed is a filter), and the results are sent to standard output (but can be redirected to a file).

sed Syntax

- sed [-n] [-e] [‘command’] [file…]
- sed [-n] [-f scriptfile] [file…]
- -n - only print lines specified with the print command (or the ‘p’ flag of the substitute (‘s’) command)
- -f scriptfile - next argument is a filename containing editing commands
- -e command - the next argument is an editing command rather than a filename, useful if multiple commands are specified

sed syntax


$ sed -e 'address command' input_file
(a) Inline Script
$ sed -f script.sed input_file
(b) Script File

Commands

- command is a single letter

- Example:

Deletion: d
[address1][,address2]d

- Delete the addressed line(s) from the pattern space; line(s) not passed to standard output.
- A new line of input is read and editing resumes with the first command of the script.

Address and Command Examples: delete

d	deletes all lines
6d	deletes line 6
/^$/d	deletes all blank lines
1,10d	deletes lines 1 through 10
1,/^$/d	deletes from line 1 through the first blank line
/^$/,$d	deletes from the first blank line through the last line of the file
/^$/,10d	deletes from the first blank line through line 10
/^ya*y/,/[0-9]$/d	deletes from the first line that begins with yay, yaay, yaaay, etc. through the first line that ends with a digit

Multiple Commands

- Braces {} can be used to apply multiple commands to an address

```
[address][,address]{
   command1
   command2
   command3
}
```

- The opening brace must be the last character on a line
- The closing brace must be on a line by itself
- Make sure there are no spaces following the braces
- Alternatively, use “;” after each command:

```
[address][,address]{command1; command2; command3; }
```

Print

- The print command (p) can be used to force the pattern space to be output, useful if the -n option has been specified
- Syntax:

[address1[,address2]]p

- Note: if the -n or #n option has not been specified, p will cause the line to be output twice!
- Examples:

1,5p will display lines 1 through 5

/^$/,$p will display the lines from the first blank line through the last line of the file

Substitute

- Syntax:

[address(es)]s/pattern/replacement/[flags]

- pattern - search pattern
- replacement - replacement string for pattern
- flags - optionally any of the following

  n - a number from 1 to 512 indicating which occurrence of pattern should be replaced

  g - global, replace all occurrences of pattern in pattern space

  p - print contents of pattern space

Substitute Examples

s/Puff Daddy/P. Diddy/

- Substitute P. Diddy for the first occurrence of Puff Daddy in pattern space

s/Tom/Fred/2

- Substitutes Fred for the second occurrence of Tom in the pattern space

s/wood/plastic/p

- Substitutes plastic for the first occurrence of wood and outputs (prints) pattern space

Append, Insert, and Change

- Syntax for these commands is a little strange because they must be specified on multiple lines
- append

[address]a\

text

- insert

[address]i\

text

- change

[address(es)]c\

text

- append/insert for single lines only, not range

Append and Insert

- Append places text after the current line in pattern space

- Insert places text before the current line in pattern space

  - each of these commands requires a \ following it
  - text must begin on the next line.
  - if text begins with whitespace, sed will discard it unless you start the line with a \

Using !

- If an address is followed by an exclamation point (!), the associated command is applied to all lines that don't match the address or address range

- Examples:

1,5!d delete all lines except 1 through 5

/black/!s/cow/horse/ substitute "horse" for "cow" on all lines except those that contained "black"

- e.g.

"The brown cow" -> "The brown horse"

"The black cow" -> "The black cow"

Regular Expressions: use with sed

Metacharacter	Description/Matches…
.	Any one character, except new line
*	Zero or more of preceding character
^	A character at beginning of line
$	A character at end of line
\char	Escape the meaning of *char* following it
[ ]	Any one of the enclosed characters
\(\)	Tags matched characters to be used later
x\{m\}	Repetition of character x, m times
\<	Beginning of word
\>	End of word

Sed – more info

- https://www.gnu.org/software/sed/manual/sed.html
- https://www.tutorialspoint.com/sed/
- http://www.grymoire.com/Unix/Sed.html
- ….

Useful examples (for all the tools):

- https://www.howtogeek.com
- https://stackoverflow.com

Hands-on 2

1. Create the file test10.log containing

   A is the first letter
   B is the second letter
   C is the third letter
   D is the fourth letter
   E is the fifth letter
   F is the sixth letter
   G is the seventh letter
   H is the eighth letter
   i is the ninth letter
   j is the tenth letter

2. Make a new file named 1.sed which has only lines 1,2,3,4,5 and 6 from the original input file
3. Make a new file which has only lines 2 and 4 removed from the original file
4. Make a new file which has the line “The end” added to the end of the file
5. Make a new file which has the line “The end” added after the line containing “tenth”
6. Make a new file which has 8th line changed into “Change”
7. Make a new file which has the 1st line changed into “A is the 1st letter”
8. Make a new file which has only lines containing at least one upper case character

1. nano test10.log
2. sed '1,6!d' test10.log > 1.sed

   or

   sed -n 1,6p test10.log > 1.sed
3. sed -e '2d;4d' test10.log > 2.sed
4. sed '$ a\ The end' test10.log > 3.sed
5. sed '/tenth/a\ The end' test10.log > 4.sed
6. sed 's/H is the eighth letter/Change/' test10.log > 5.sed

   or

   sed -e '/eighth/i\Change' test10.log|sed 9d > 5.sed

   or

   sed 8c\Change test10.log > 5.sed
7. sed '/first/c\A is the 1st letter' test10.log > 6.sed

   or

   sed s/first/1st/ test10.log > 6.sed
8. sed '/[A-Z]/!d' test10.log > 7.sed

awk

- Answers the question:
  - What do I do if I want to search for a pattern and actually use it?
- Combination of grep and other commands
  - Searches for some pattern or condition and performs some command on it
- Complete programming language
  - Looks a lot like C syntactically
  - Variables are declared bash style

When Would You Want To Use awk?

- Whenever you want to search for some pattern and perform some action

- Example: I want to go through and calculate the average score on the Midterm

Advantages of Awk

- awk is an interpreted language so you can avoid the usually lengthy edit-compile-test-debug cycle of software development.
- Can be used for rapid prototyping.
- The awk language is very useful for producing reports from large amounts of raw data, such as summarizing information from the output of other utility programs like ls.

Combining Expression Options

- ||
  - Or
- &&
  - And
- \(\)
  - Grouping

Action Commands

- -print
  - Simply prints out the name of the file that was found
  - Most common action
- -exec
  - Executes a command
- -ok
  - Executes a command, but prompts the user first

sed vs. awk

- **sed** is a pattern-action language, like **awk**
- awk processes fields while sed only processes lines
- sed +
  - regular expressions
  - fast
  - concise
- sed –
  - hard to remember text from one line to another
  - not possible to go backward in the file
  - no way to do forward references like /..../+1
  - no facilities to manipulate numbers
  - cumbersome syntax
- awk +
  - convenient numeric processing
  - variables and control flow in the actions
  - convenient way of accessing fields within lines
  - flexible printing
  - C-like syntax

A few basic things about awk

- awk reads from a file or from its standard input, and outputs to its standard output.
- awk recognizes the concepts of "file", "record" and "field".
- A file consists of records, which by default are the lines of the file. One line becomes one record.
- awk operates on one record at a time.
- A record consists of fields, which by default are separated by any number of spaces or tabs.
- Field number 1 is accessed with $1, field 2 with $2, and so forth. $0 refers to the whole record.

Program Structure in Awk

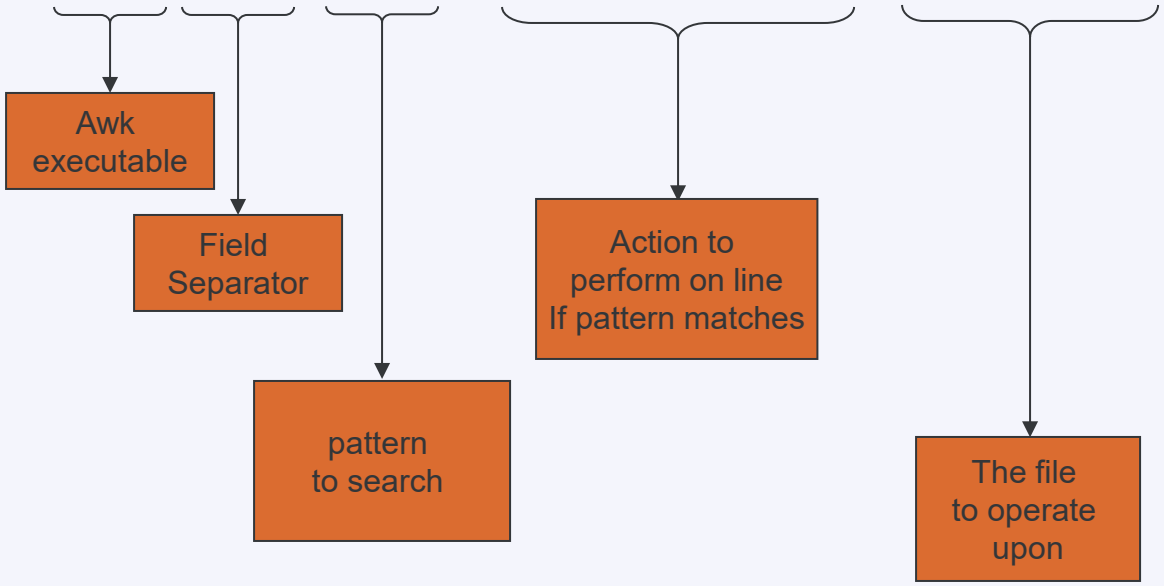
- An awk program is a sequence of statements of the form:

```
pattern    { action }
pattern    { action }
...
```

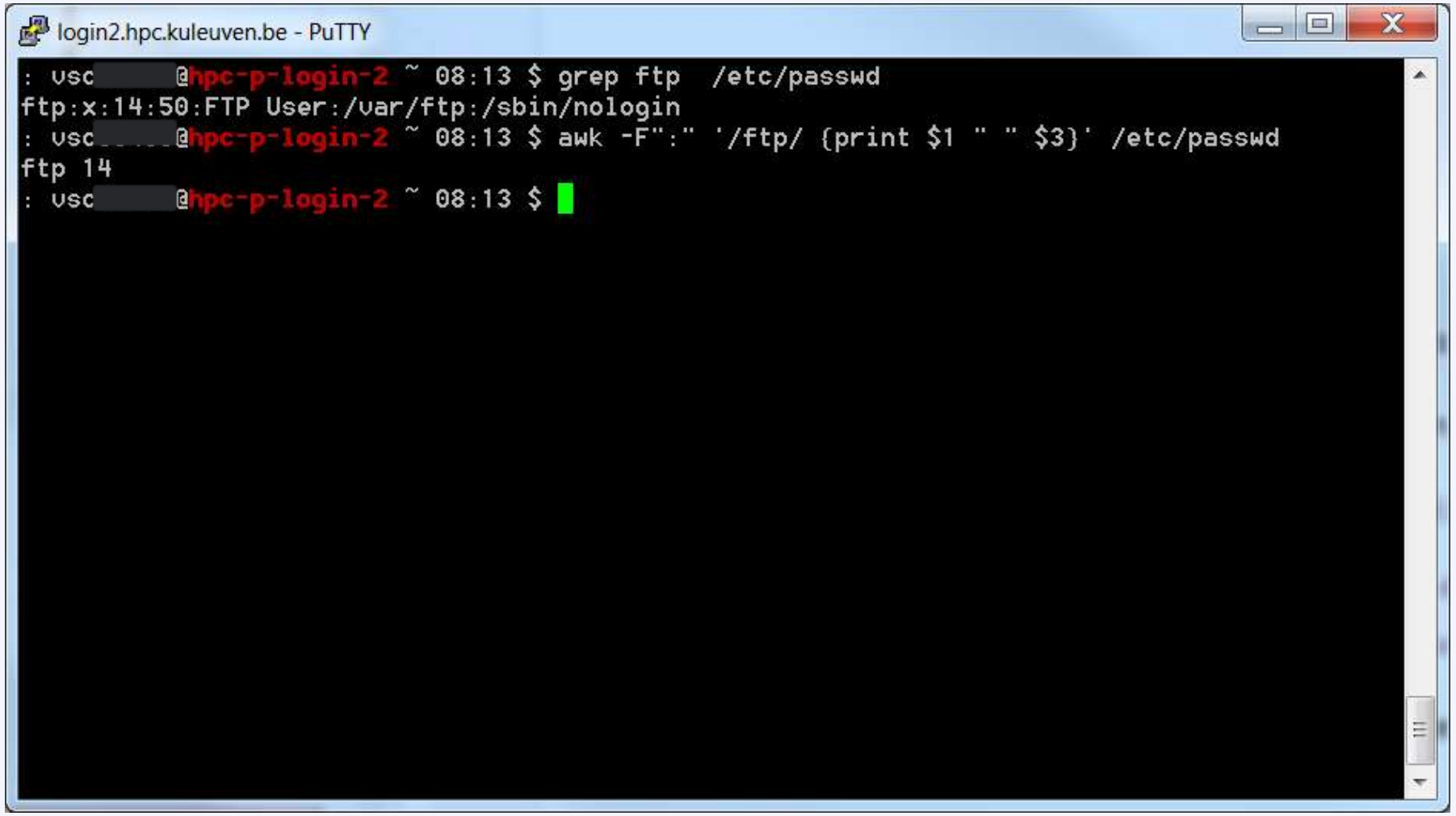
- pattern in front of an action acts as a selector that determines whether the action is to be executed.
- Patterns can be : regular expressions, arithmetic relational expressions, string-valued expressions, and arbitrary boolean combinations of these.
- action is a sequence of action statements terminated by newlines or semicolons.

A simple example

```
$ awk –F”:” ‘/ftp/ {print $1 “ “ $3}’ /etc/passwd
```



A simple example (cont..)



Running awk programs

There are four ways in which we can run awk programs

- One-shot: Running a short throw-away awk program.

```
$ awk 'program' input-file1 input-file2
```

  ... where *program* consists of a series of patterns and actions.

- Read Terminal: Using no input files (input from terminal instead).

```
$ awk 'program' <ENTER>
<input lines>
<input lines>
ctrl-d
```

Running awk programs

There are four ways in which we can run awk programs

- Long: Putting permanent awk programs in files.

```
$ awk -f source-file input-file1 input-file2 ...
```

- Executable: self-contained awk scripts, using the `#!' script mechanism.

  Self-contained awk scripts are useful when you want to write a program which users can invoke without their having to know that the program is written in awk.

Basic awk Syntax

- `awk [options] 'script' file(s)`

- `awk [options] -f scriptfile file(s)`

Options:

-F to change input field separator

-f to name script file

Basic awk Program

- consists of patterns & actions:

```
pattern {action}
```

  - if pattern is missing, action is applied to all lines
  - if action is missing, the matched line is printed
  - must have either pattern or action

Example:

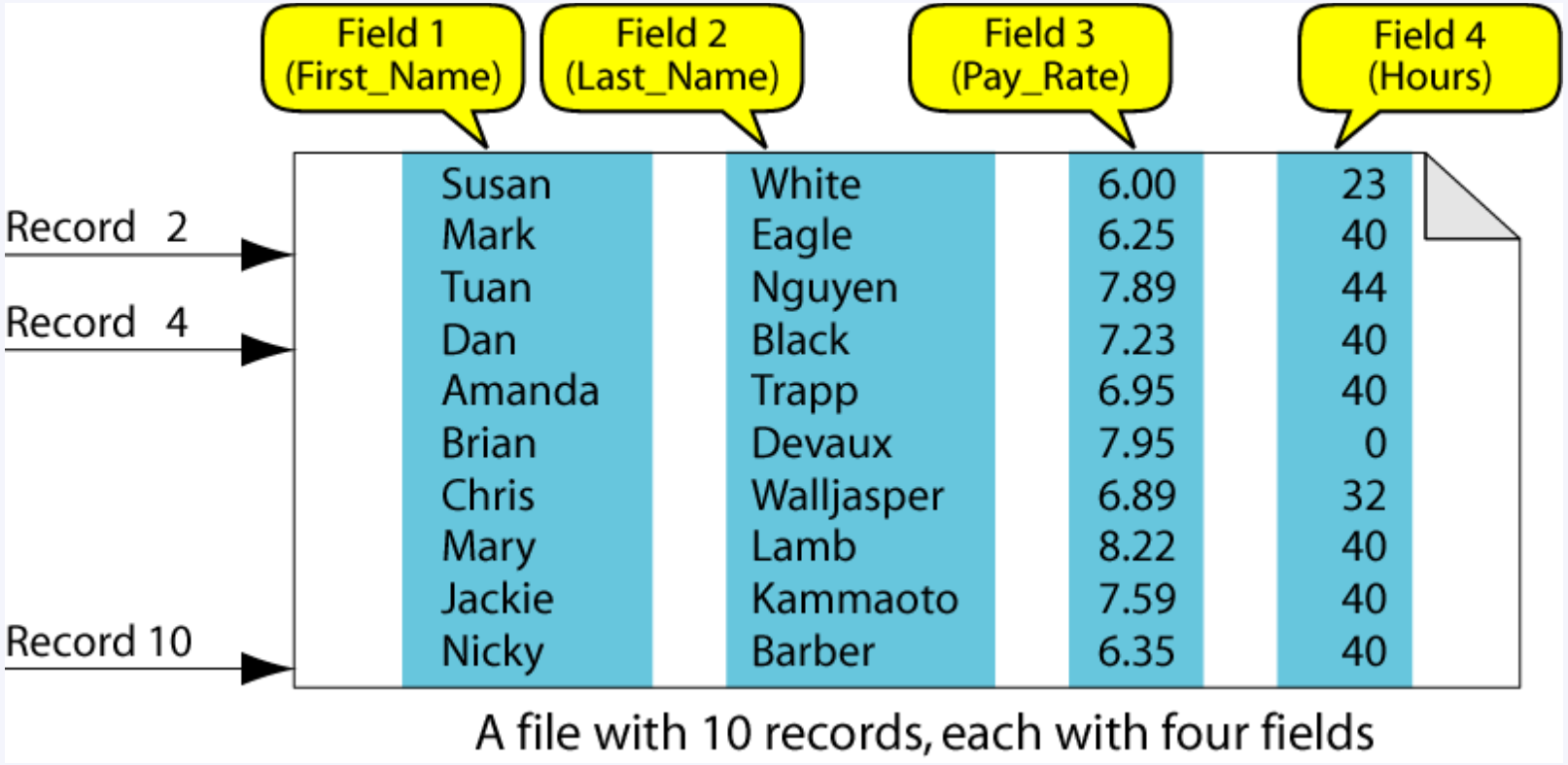
```
awk '/for/' testfile
```

- prints all lines containing string “for” in testfile

Basic Terminology: input file

- A field is a unit of data in a line
- Each field is separated from the other fields by the field separator
  - default field separator is whitespace
- A record is the collection of fields in a line
- A data file is made up of records

Example Input File



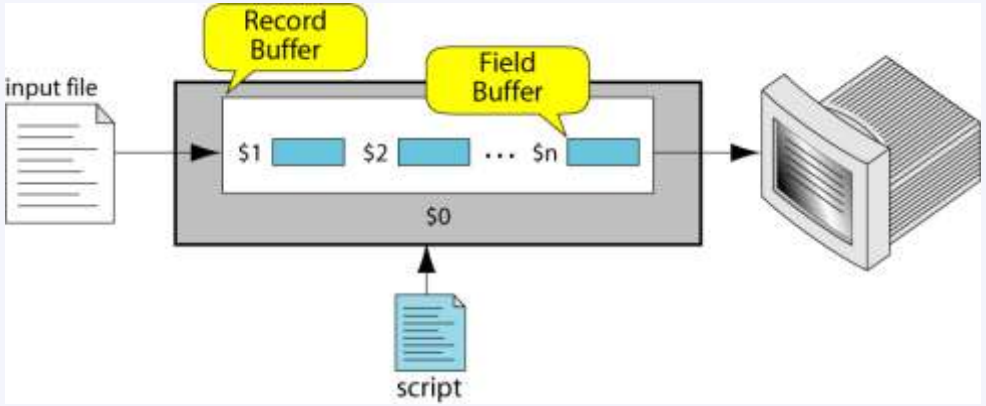
A file with 10 records, each with four fields

Buffers

- awk supports two types of buffers:
  record and field

- field buffer:
  - one for each fields in the current record.
  - names: $1, $2, …

- record buffer :
  - $0 holds the entire record



Some System Variables

FS	Field separator (default=whitespace)
RS	Record separator (default=\n)
NF	Number of fields in current record
NR	Number of the current record
OFS	Output field separator (default=space)
ORS	Output record separator (default=\n)
FILENAME	Current filename

Example: Records and Fields

```
$ cat emps
Tom Jones          4424     5/12/66 543354
Mary Adams         5346     11/4/63 28765
Sally Chang        1654     7/22/54 650000
Billy Black        1683     9/23/44 336500

$ awk '{print NR, $0}' emps
1 Tom Jones        4424     5/12/66 543354
2 Mary Adams       5346     11/4/63 28765
3 Sally Chang      1654     7/22/54 650000
4 Billy Black      1683     9/23/44 336500
```

Example: Space as Field Separator

```
$ cat emps
Tom Jones          4424     5/12/66 543354
Mary Adams         5346     11/4/63 28765
Sally Chang        1654     7/22/54 650000
Billy Black        1683     9/23/44 336500

$ awk '{print NR, $1, $2, $5}' emps
1 Tom Jones 543354
2 Mary Adams 28765
3 Sally Chang 650000
4 Billy Black 336500
```

Example: Colon as Field Separator

```
$ cat em2
Tom Jones:4424:5/12/66:543354
Mary Adams:5346:11/4/63:28765
Sally Chang:1654:7/22/54:650000
Billy Black:1683:9/23/44:336500

$ awk -F: '/Jones/{print $1, $2}' em2
Tom Jones 4424
```

Advanced awk features

- Awk borrows a lot from the C language.
- The if loop, for loop and while loop have the same constructs as in C.
- Awk's variables are stored internally as strings.

```
eg. x = "1.01"
     x = x + 1
     print x
```

The above will print the value 2.01

- Comparison operators in awk are : `"==", "<", ">", "<=", ">=", "!=", "~" and "!~"`.
- **"~"** and **"!~"** operators mean "matches" and "does not match".

Awk Examples

```
$ awk '{ print $0 }' /etc/passwd
```

Prints all the lines in /etc/passwd

```
$ awk -F":" '{ print "username: " $1 "\t\tuid:" $3" }' /etc/passwd
```

Prints the 1st and 3rd fields of each line in /etc/passwd. The fields are separated by ":"

```
$ awk -f script1.awk /etc/passwd
```

script1.awk

```
BEGIN{ x=0 }# The BEGIN block is executed before processing the file
/^$/ { x=x+1 } # For every null line increment the count
END { print "I found " x " blank lines. :)" } #Executed at the end
```

The above script calculates the number of null lines. Note that BEGIN and END are special patterns.

Awk examples (cont..)

```
$ awk 'BEGIN { RS = "/" } ; { print $0 }' file1.txt
```

RS is the record separator (default is \n). In this example the RS is modified to “/” and then the file is processed. So awk will distinguish between records by “/” character.

```
$ awk '$1 ~ /foo/ { print $0 }' file.txt
```

The pattern will print out all records from file file.txt whose first fields contain the string “foo”.

```
$ awk '{ print $(2*2) }' file.txt
```

In the above example the field number is an expression. So awk will print the 4th fields of all the records.

Awk examples (cont..)

```
$ awk '{ $3 = $2 - 10; print $2, $3 }' inventory-shipped
```

This example will subtract the second field of each record by 10 and store it in the third field.

```
$ awk 'BEGIN { FS = "," } ; { print $2 }' file.txt
```

FS is the field separator in awk. In the above example we are asking awk to separate the fields by “,” instead of default “ “.

```
$ awk 'BEGIN { OFS = ";"; ORS = "\n\n" }
                { print $1, $2 }'  file1.txt
```

OFS is the Output field Separator, ORS is Output record separator. This prints the first and second fields of each input record separated by a semicolon, with a blank line added after each line.

Awk examples

Consider that we have the following input in a file called grades

```
john 85 92 78 94 88
andrea 89 90 75 90 86
jasper 84 88 80 92 84
```

The following awk script will find the average

```
# average five grades
{ total = $2 + $3 + $4 + $5 + $6
  avg = total / 5
  print $1, avg }
```

```
$ awk -f grades.awk grades
```

Awk examples (cont..)

```
$ awk 'BEGIN { OFMT = "%d"  # print numbers as integers
               print 17.23 }‘
```

This will print 17. OFMT is the output format specifier.

```
$ awk –f mailerr.awk
  { report = "mail bug-system"
    print "Awk script failed:", $0 | report
    print "at record number", FNR, "of", FILENAME  | report
    close(report)
  }
```

This script opens a pipe to the mail command and prints output into the pipe.

When the pipe is closed the mail is sent. Awk assumes that whatever comes after the “|” symbol is a command and creates a process for it.

Expression Pattern types

- match
  - entire input record
    regular expression enclosed by ‘/’s
  - explicit pattern-matching expressions
    ~ (match), !~ (not match)

- expression operators
  - arithmetic
  - relational
  - logical

Example: match input record

```
% cat employees2
Tom Jones:4424:5/12/66:543354
Mary Adams:5346:11/4/63:28765
Sally Chang:1654:7/22/54:650000
Billy Black:1683:9/23/44:336500

% awk -F: '/00$/' employees2
Sally Chang:1654:7/22/54:650000
Billy Black:1683:9/23/44:336500
```

Arithmetic Operators

Operator	Meaning	Example
+	Add	x + y
-	Subtract	x – y
*	Multiply	x * y
/	Divide	x / y
%	Modulus	x % y
^	Exponential	x ^ y

Example:

```
% awk '$3 * $4 > 500 {print $0}' file
```

Relational Operators

Operator	Meaning	Example
<	Less than	x < y
<=	Less than or equal	x < = y
==	Equal to	x == y
!=	Not equal to	x != y
>	Greater than	x > y
>=	Greater than or equal to	x > = y
~	Matched by reg exp	x ~ /y/
!~	Not matched by req exp	x !~ /y/

Logical Operators

Operator	Meaning	Example
&&	Logical AND	a && b
||	Logical OR	a || b
!	NOT	! a

Examples:

```
$ awk '($2 > 5) && ($2 <= 15){print $0}' file
$ awk '$3 == 100 || $4 > 50' file
```

awk variables

- A user can define any number of variables within an awk script
- The variables can be numbers, strings, or arrays
- Variable names start with a letter, followed by letters, digits, and underscore
- Variables come into existence the first time they are referenced; therefore, they do not need to be declared before use
- All variables are initially created as strings and initialized to a null string “”

awk variables

Format:

```
variable = expression
```

Examples:

```
$ awk '$1 ~ /Tom/ wage = $3 * $4; print wage}' filename
$ awk '$4 == "CA" {$4 = "California"; print $0}' filename
```

awk assignment operators

=	assign result of right-hand-side expression to left-hand-side variable
++	Add 1 to variable
--	Subtract 1 from variable
+=	Assign result of addition
-=	Assign result of subtraction
*=	Assign result of multiplication
/=	Assign result of division
%=	Assign result of modulo
^=	Assign result of exponentiation

Output Statements

`print`

print easy and simple output

`printf`

print formatted (similar to C printf)

`sprintf`

format string (similar to C sprintf)

Function: print

- Writes to standard output
- Output is terminated by ORS
  - default ORS is newline
- If called with no parameter, it will print $0
- Printed parameters are separated by OFS,
  - default OFS is blank
- Print control characters are allowed:
  - \n \f \a \t \\ …

Redirecting print output

- Print output goes to standard output unless redirected via:
  - > “file”
  - >> “file”
  - | “command”

- will open file or command only once
- subsequent redirections append to already open stream

print example

```
$ awk '{print}' grades
john 85 92 78 94 88
andrea 89 90 75 90 86

$ awk '{print $0}' grades
john 85 92 78 94 88
andrea 89 90 75 90 86

$ awk '{print($0)}' grades
john 85 92 78 94 88
andrea 89 90 75 90 86
```

print example

```
$ awk '{print $1, $2}' grades
john 85
andrea 89

$ awk '{print $1 "," $2}' grades
john,85
andrea,89
```

print example

```
$ awk '{OFS="-";print $1 , $2}' grades
john-85
andrea-89

$ awk '{print $1 "," $2}' grades
john,85
andrea,89
```

print example

```
$ awk '{print $1 , $2 > "file"}' grades

$ cat file
john 85
andrea 89
jasper 84
```

print example

```
$ awk '{print $1,$2 | "sort"}' grades
andrea 89
jasper 84
john 85

$ awk '{print $1,$2 | "sort -k 2"}' grades
jasper 84
john 85
andrea 89
```

printf: Formatting output

Syntax:

```
printf(format-string, var1, var2, …)
```

- works like C printf
- each format specifier in “format-string” requires argument of matching type

Format specifiers

%d, %i	decimal integer
%c	single character
%s	string of characters
%f	floating point number
%o	octal number
%x	hexadecimal number
%e	scientific floating point notation
%%	the letter “%”

Format specifier examples

Given: x = ‘A’, y = 15, z = 2.3, and $1 = Bob Smith	
Printf Format Specifier	**What it Does**
%c	***printf("The character is %c \n", x)*** **output: The character is A**
%d	***printf("The boy is %d years old \n", y)*** **output: The boy is 15 years old**
%s	***printf("My name is %s \n", $1)*** **output: My name is Bob Smith**
%f	***printf("z is %5.3f \n", z)*** **output: z is 2.300**

Format specifier modifiers

- between “%” and letter

  %10s

  %7d

  %10.4f

  %-20s

- meaning:
  - width of field, field is printed right justified
  - precision: number of digits after decimal point
  - “-” will left justify

sprintf: Formatting text

Syntax:

```
sprintf(format-string, var1, var2, …)
```

- Works like printf, but does not produce output
- Instead it returns formatted string

Example:

```
{
 text = sprintf("1: %d - 2: %d", $1, $2)
 print text
}
```

awk builtin functions

`tolower(string)`

- returns a copy of string, with each upper-case character converted to lower-case. Nonalphabetic characters are left unchanged.

Example: tolower("MiXeD cAsE 123")

returns "mixed case 123"

`toupper(string)`

- returns a copy of string, with each lower-case character converted to upper-case.

awk references

- https://www.gnu.org/software/gawk/manual/gawk.html
- http://www.grymoire.com/Unix/Awk.html
- https://www.tutorialspoint.com/awk/
- https://www.gnu.org/software/gawk/manual/gawk.pdf
- Sed and Awk 2nd Edition (O'reilly) http://linux.iingen.unam.mx/pub/Documentacion/Shell-Bash/OReilly%20-%20Sed%20&%20Awk%202nd%20Edition.pdf

Hands-on 3

1. Download the file https://github.com/hpcleuven/Linux-tools/blob/master/lp_hpcinfo and print the number of its lines with awk
2. Download the file https://github.com/hpcleuven/Linux-tools/blob/master/lp_sys and print only the lines that contain vsc30468
3. From lp_sys print only the lines that contain both vsc30468 and 2045
4. From lp_sys print only column 3 and 4 of the lines from 30th line on
5. From lp_sys print only column 7 and the last column.
6. Print only the lines where the 3rd column has values higher than 20631950

```
1. awk 'END{print NR}' lp_hpcinfo
2. awk '/vsc30468/ {print $0}' lp_sys
3. awk '/vsc30468/ && /2045/ {print $0}' lp_sys
4. awk 'NR > 30 { print $3 ',' $4}'  lp_sys
    awk 'NR > 30 { print $3 ", " $4}'  lp_sys
5. awk '{print $7 ',' $NF}' lp_sys
6. awk '$3 > 20631950' lp_sys
```

Gnuplot

- Versatile visualization tool
  - Command driven, interactive function plotting program
- Designed for mathematics visualization
  - Student programmers looking to visualize classroom concepts
  - Open source code
- Multiple Platforms
  - Unix/Linux, Macs, lots of others
  - Windows version has a menu-driven interface
- Copyrighted but freely distributable
- Originally developed by Colin Kelley and Thomas Williams in 1986 to plot functions and data files on a variety of terminals.

Gnuplot - advantages

- Quality 2-D/3-D plots of functions and data
- Small set of commands - easy to learn
- Very transferable
  - Multiple platforms
  - Text file formats for input and command files
- Can be automated (allows it to be run by other programs)
- Can manipulate data with mathematical functions
- Existing support community
- Free! Free! Free!
  - www.gnuplot.info (gnuplot central)

Gnuplot - disadvantages

- Limited types of plots
- Contour plot process a bit difficult
- ‘Programming’ style assumed
- Limited set of styles for lines and points
- Limited text abilities for titles and labels
  - Greek letters and symbols difficult/not possible
  - Formatted math formulas difficult/not possible
- Updates/development depends on volunteers

Gnuplot

- Console Window
  - Command line prompt gnuplot>
  - Menu selections for Windows version
- Plotting Window
- Help
  - ?
  - help <topic>
- Lines beginning with “#” are comments lines and are ignored.

Gnuplot – function plots

- 2-D variable is x

```
gnuplot> plot sin(x)
```

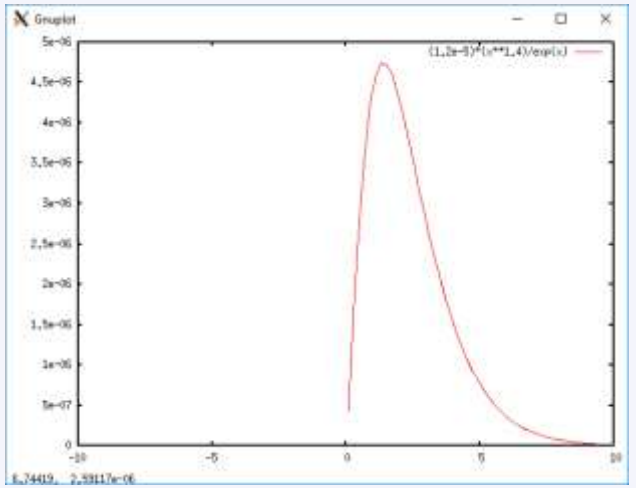
- Use replot to display a second function

```
gnuplot> replot cos(x)
```

- Math functions in fortran style

```
gnuplot> plot (1.2e-5)*(x**1.4)/exp(x)
```

- Many functions (use '`help functions`' to list)
  - e.g. abs() sinh() log() rand() acos() sqrt()

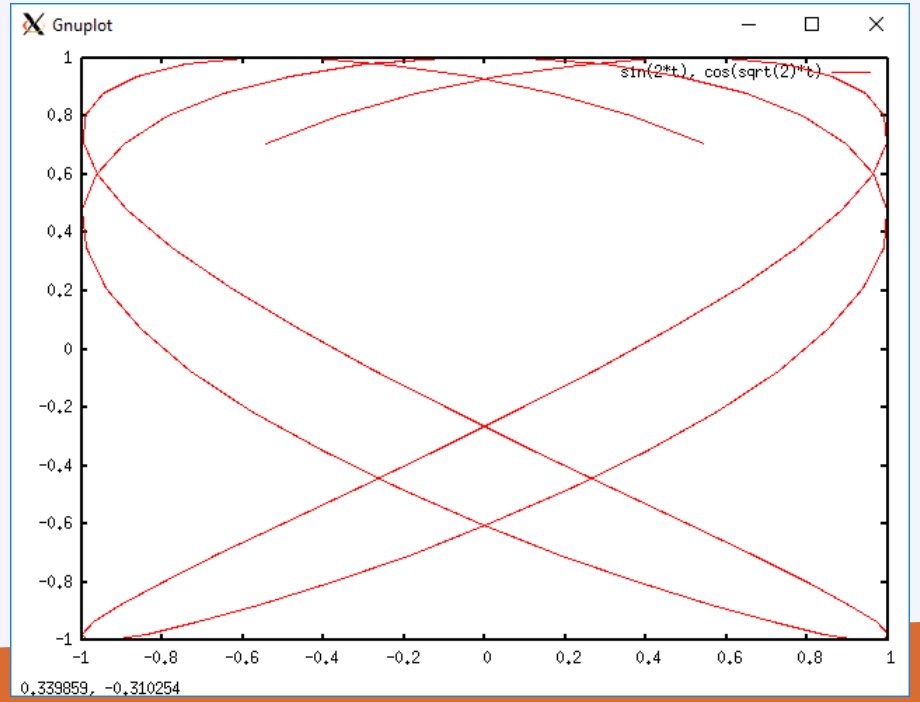


Gnuplot – parametric plots

```
gnuplot> set parametric
```

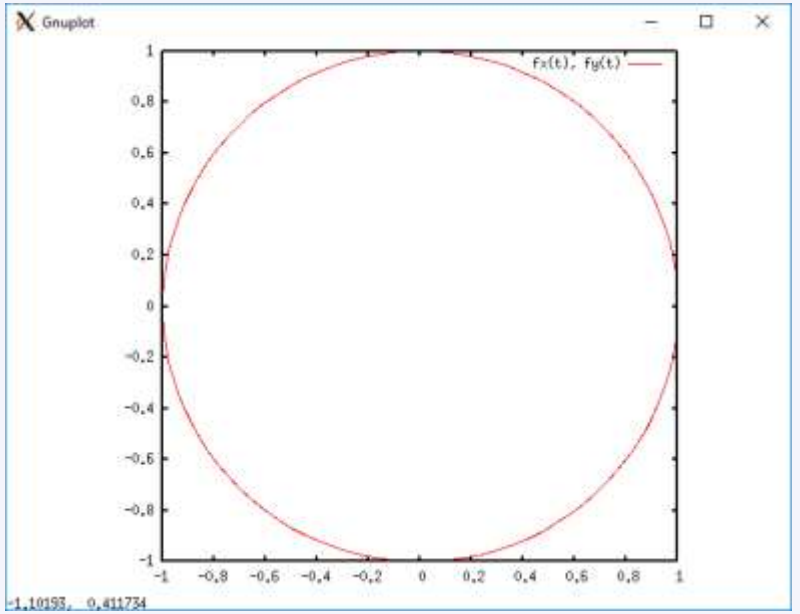
- 2-D “dummy” variable is t

```
gnuplot> plot sin(2*t), cos(sqrt(2)*t)
gnuplot> unset parametric
```



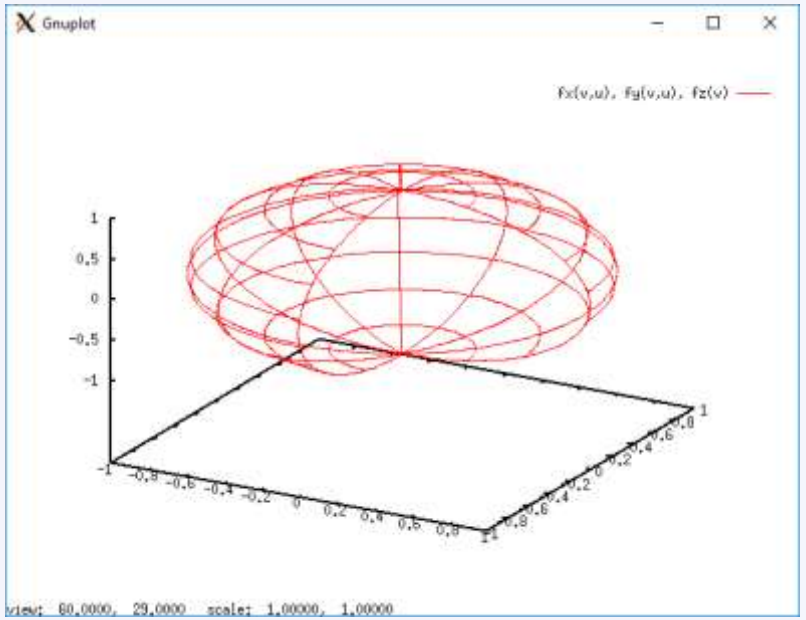
Gnuplot – parametric plots

```
gnuplot> set size square
gnuplot> r=1
gnuplot> fx(t) = r*cos(t)
gnuplot> fy(t) = r*sin(t)
gnuplot> plot fx(t),fy(t)
```



Gnuplot – 3D plots

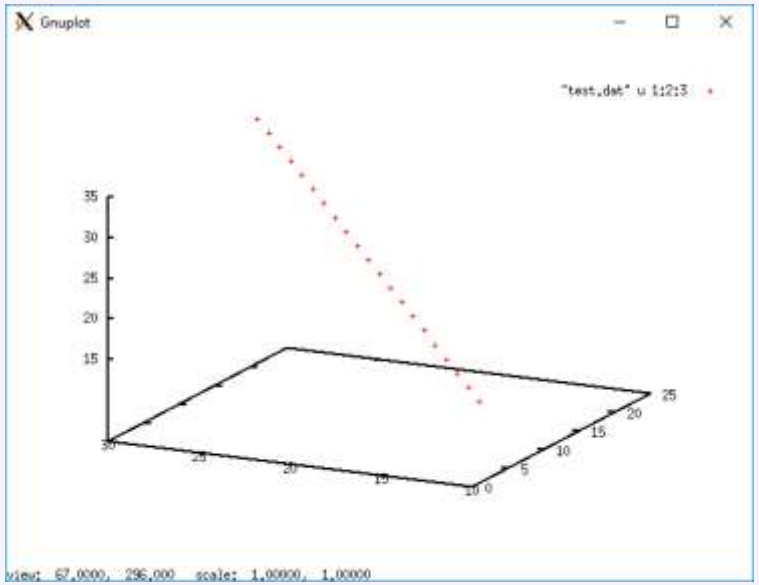
```
gnuplot> set parametric
gnuplot> r=1
gnuplot> fx(v,u) = r*cos(v)*cos(u)
gnuplot> fy(v,u) = r*cos(v)*sin(u)
gnuplot> fz(v)   = r*sin(v)
gnuplot> splot fx(v,u),fy(v,u),fz(v)
```

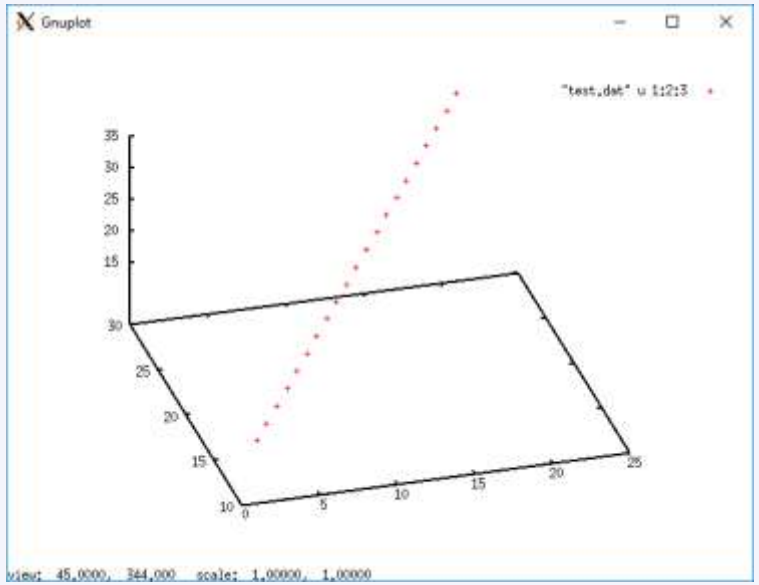


Gnuplot – 3D plots

```
gnuplot> splot "test.dat" u 1:2:3
```

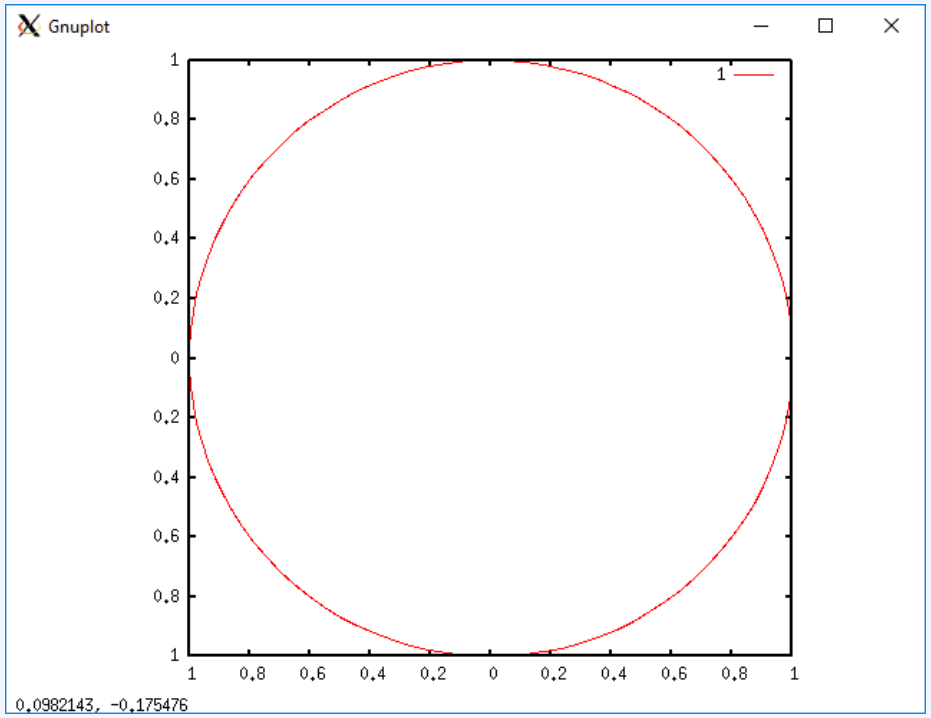
For 3d graphs splot, the view and scaling of the graph can be changed with mouse buttons





Gnuplot – polar plots

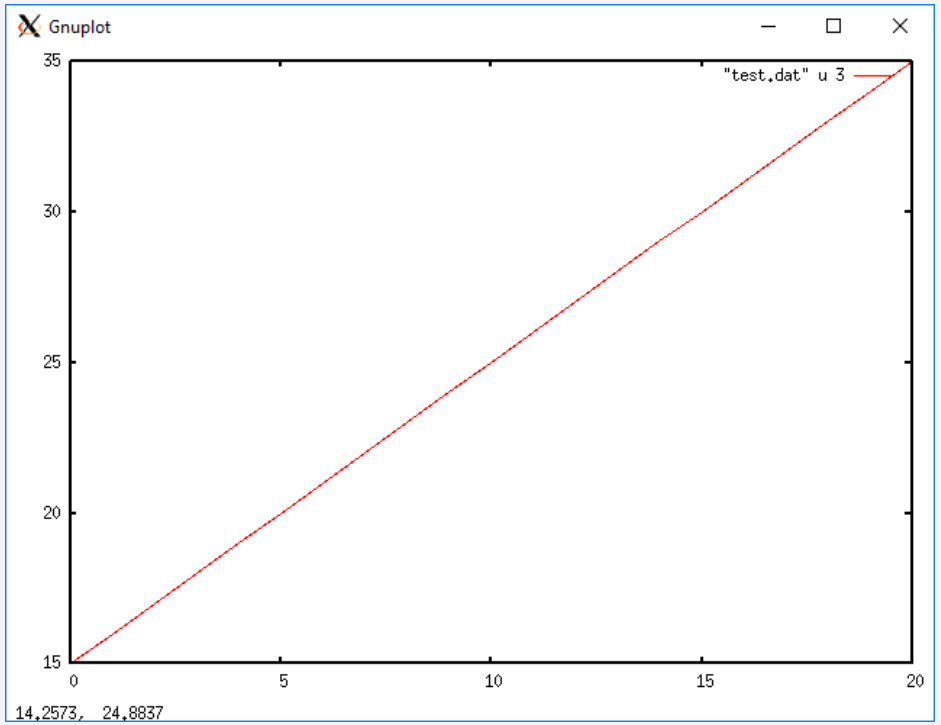
```
gnuplot> set polar
gnuplot> set size ratio -1
gnuplot> plot 1
gnuplot> unset polar
```

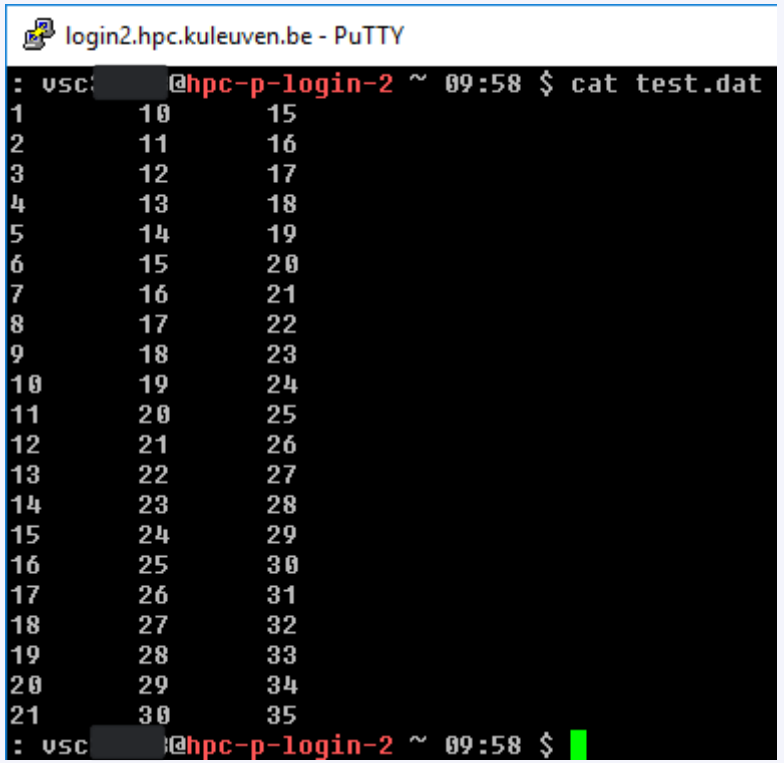


Gnuplot - data plots

Single channel

```
gnuplot> plot "test.dat" u 3 w l
gnuplot> plot "test.dat" using 3 with lines
```

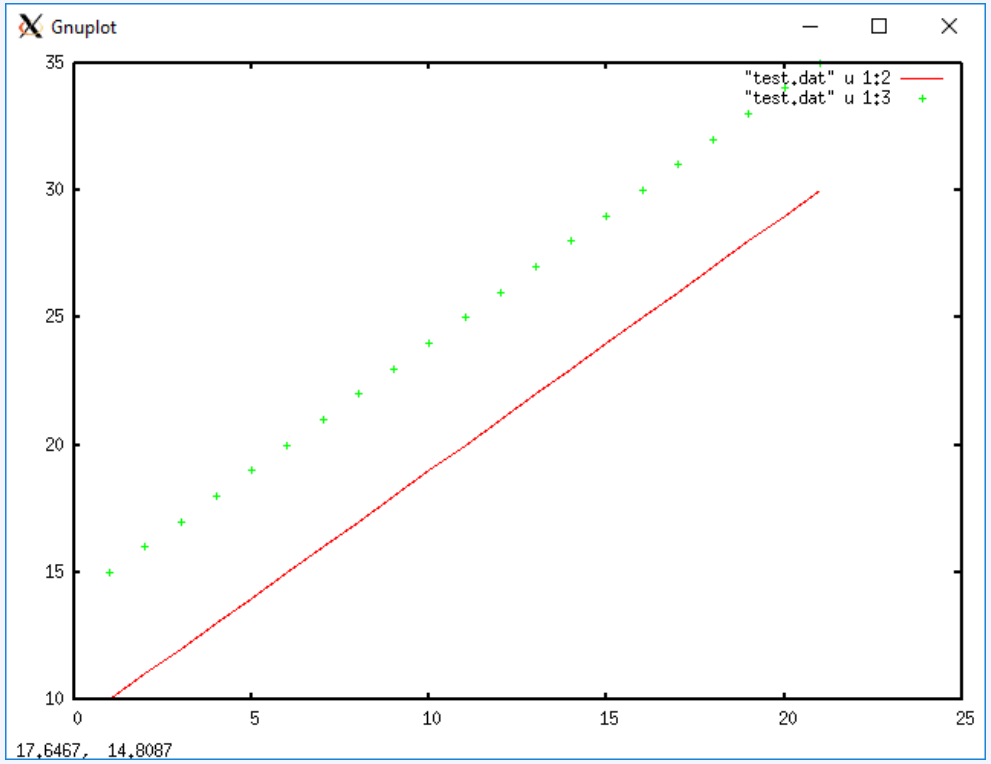


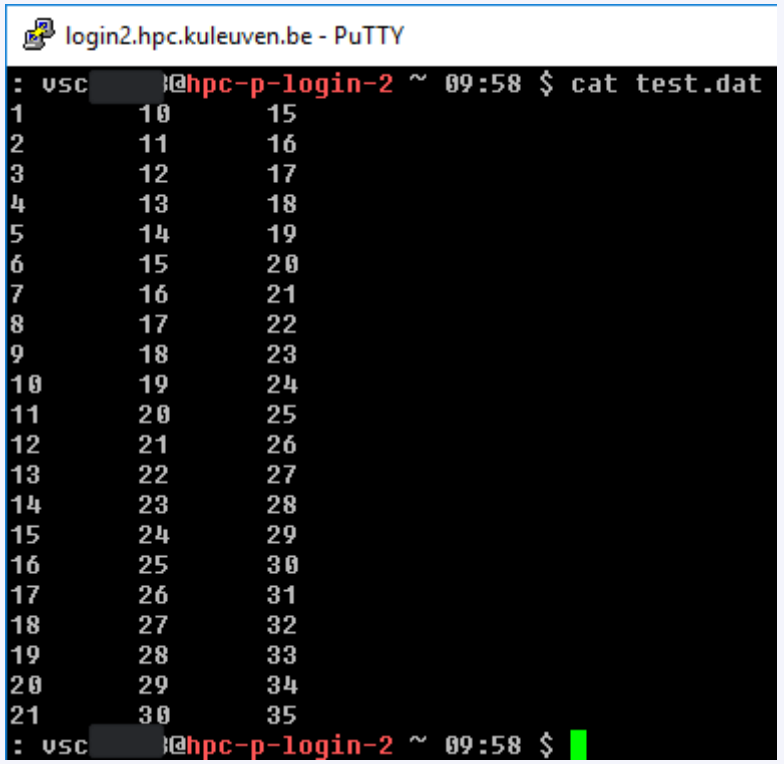


Gnuplot - data plots

Cross plots

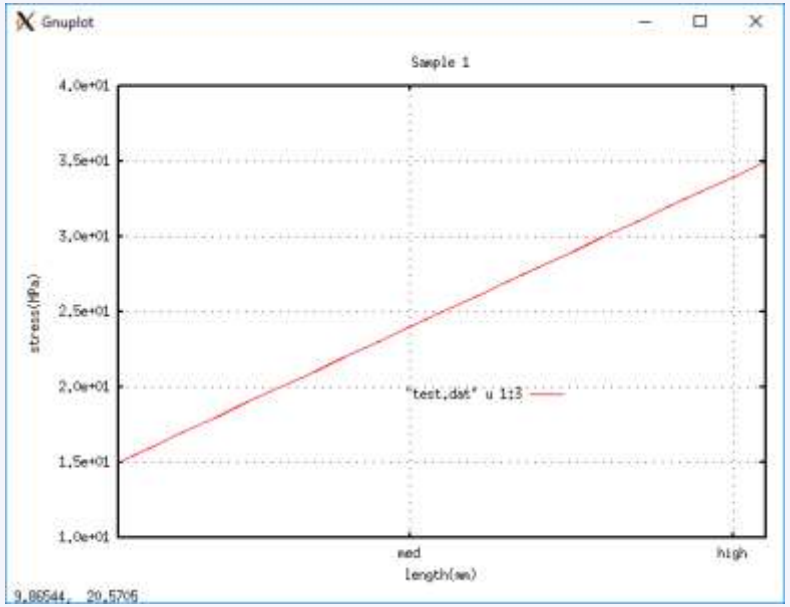
```
gnuplot> plot "test.dat" u 1:2 w l, "test.dat" u 1:3 w p
```





Gnuplot - settings

- `gnuplot> plot [:] [10:40]  "test.dat" u 1:3 w l`
- `gnuplot> set xlabel 'length(mm)'; set ylabel 'stress(MPa)' -1,0; replot`
- `gnuplot>  set title "Sample 1"; set key 15,20; replot`
- `gnuplot>  set xtics ('low' 0, 'med' 10,'high' 20); set grid; set format y '%.1e'; replot`



Plot with the new settings

Gnuplot - settings

- Relative Graph size: `> set size 1,1`
- Function sampling pts: `> set samples 1000`
- Polar Coordinates: `> set polar`
- Contour plots: `> set contour base`
- Perspective: `> set view 90,0`
- Angle Units: `> set angles degrees`
- Style: `> set data style lines`

Gnuplot – basic functions

command line entry	description of action
exit or quit	Quits gnuplot
help or ? <topic>	help or help on a specific topic
set key or unset key	Adds or removes legend from display
set xrange [a:b]	Controls the range of values on the x axis
plot sin(x**2-1) with points <value>	plots sin(x**2 – 1) - lots of in-built math functions - <value> between 1 and 8 denotes different pointstyles
plot sin(x) with lines <value>	<value> between 1 and 8 denotes different linestyles
plot 'test.dat' u 1:2 t 'test' w l 1,'test.dat' u 1:3 t 'test' w p 2	plots multiple sets of data on same graph: Curve of cols 1 and 2 from <test.dat> plus points from cols 1 and 3 of <test.dat>. The t is an abbr. of title for labelling in the legend
set term postscript set term png	Output in postscript/png file rather than to the display
set output 'filename.eps'	Sets up an output file for output to go to
replot	redisplays plot with new settings
set term x11	sets output back to the screen, rather than a file

Gnuplot - settings

- place or hide key

```
set key top center, unset key
```

- set a title

```
set title "the title"
```

- define axis labels

```
set xlabel "x [pc]", set ylabel "y [pc]"
```

- change the number format

```
set format x "%10.3f"
```

- plot an arrow

```
set arrow from 0.5,0 to 0.5,1
```

- define a label

```
set label "rarefaction wave" at 0.5,0
```

- set border style

```
set border lw 3
```

Gnuplot - settings

- color, width and shape of lines/points

  ```
  linetype / lt, pointtype / pt,
  linewidth / lw, pointsize / ps
  ```

- logscale

  ```
  [un]set logscale [xy],
  ```

- select zoom

  ```
  set xrange [0:10] ... manually selected range of x-axis,
  set yrange [*:*] ... select zoom of y-axis automatically,
  set autoscale ... select zoom of any axis automatically
  ```

Gnuplot

- Show command gives current settings

```
gnuplot> show xlabel
```

- Reset recalls default settings
- Save records current settings and plot statement

```
gnuplot> save '/home/mag/test.gp'
```

- Load executes settings from a file

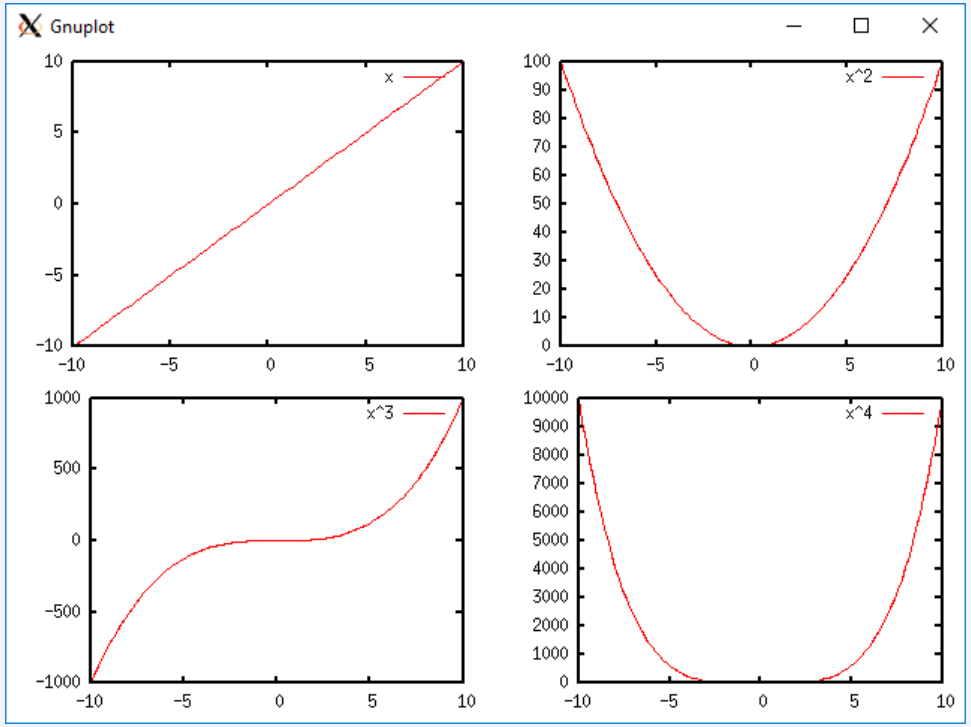
```
gnuplot> load '/home/mag/test.gp'
```

- Help is extremely useful in determining set syntax

```
gnuplot> help set
```

Gnuplot - multiplot

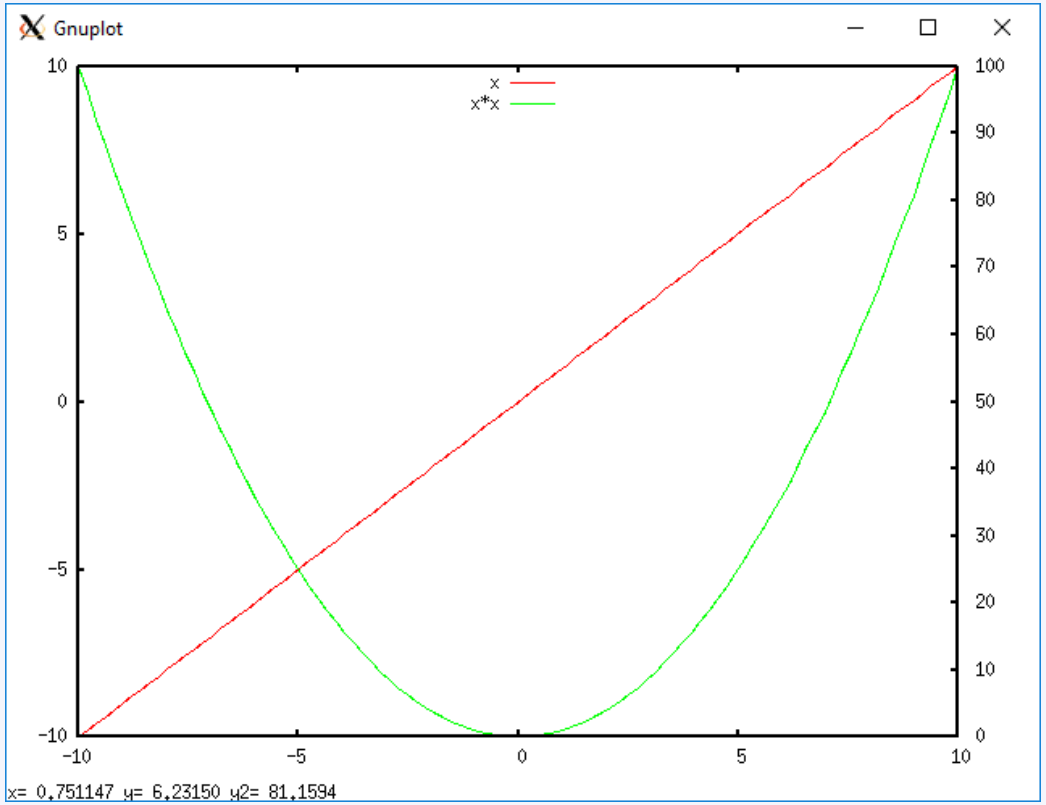
- stack several plot commands
  `set multiplot`
- scale the plot
  `set size`
- place the plot
  `set origin`
- leave multiplot mode
  `unset multiplot`




```
set multiplot
set origin 0 ,0
set size 0.5 ,0.5
plot x*x*x t 'x^3'
set origin 0 ,0.5
plot x t 'x'
set origin 0.5 ,0.5
plot x*x t 'x^2'
set origin 0.5 ,0
plot x*x*x*x t 'x^4'
```

Gnuplot - multiple graphs - y1 and y2 axis

```
set ytics nomirror
set y2tics 0, 10
set key top center
plot x axis x1y1, x*x axis x1y2
```



Gnuplot - animations

- If you use gnuplot 4.6 you can use a do loop:

  If you want to store the snapshots use:

```
filename(n) = sprintf("file_%d", n)
plot for [i=1:10] filename(i) using 1:2 with lines
```

- In earlier versions you have to place your code in two files and to use “reread”:

File 1: `t=0; tmax=18;load "file2.gnuplot"`

File 2:

```
t=t+1;outfile = sprintf('view%03.0f.png',t);
set view t*5,30,1,1;set output outfile; plot data.txt;
if(t<tmax) reread;
```

Gnuplot - Fitting data

- Define the power law

```
d(x)=c+a*sqrt(b*x)
```

- Fit your data - - you might need to set initial values for a,b,c

```
fit d(x) 'cl1d_integrals.dat' u 1:5 via a,b,c
print a,b
plot [0:5000] c+a*sqrt(x*b) t '' w l `test1.dat' u 1:5 t '' w l
```

- Drawing error bars

```
plot ’data.txt’ u 0:1:2:3 w yerrorbars
```

  Uses column 1 as y values, column 2 as lower end of the vertical error bar and column 3 as the upper end of the vertical error bar.

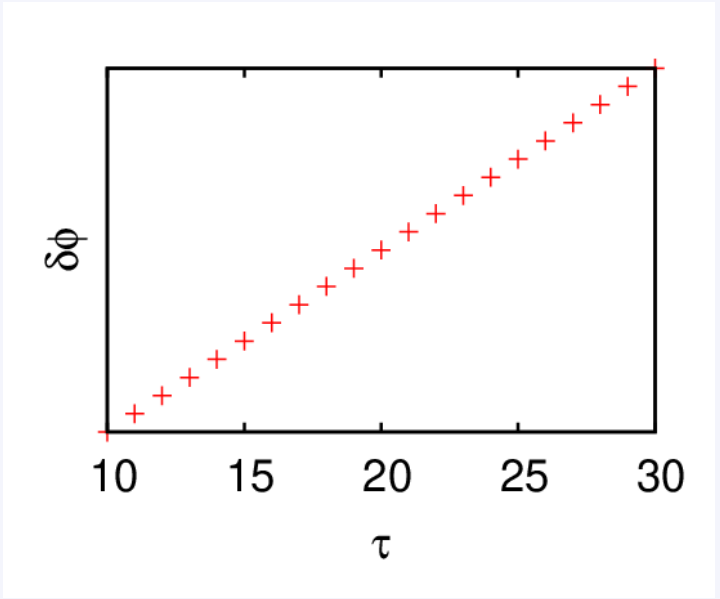
- Horizontal error bars are created with `xerrorbars`

Gnuplot and LaTeX

Gnuplot is easy to produce postscript plots that an be used by scientific publications in LaTeX

- Change the terminal type
- Possibility to use LaTeX notation

```
set terminal postscript eps size 3.5,2.62 \
enhanced color font 'Helvetica,30' linewidth 2
set output 'test.eps'
set xlabel '{/Symbol t}'
set ylabel '{/Symbol d}{/Symbol f}'
unset ytics
set title ''
unset label
set pointsize 2
plot 'test.dat' u 2:3 t ""
reset
```



Gnuplot and LaTeX

```
gnuplot> set terminal latex
gnuplot> set output "example1.tex"
gnuplot> plot [-3.14:3.14] sin(x)
```

- Results in example1.tex

```
% GNUPLOT: LaTeX picture
\setlength{\unitlength}{0.240900pt}
\ifx\plotpoint\undefined\newsavebox{\plotpoint}\fi\
begin{picture}(1500,900)(0,0)
\sbox{\plotpoint}{\rule[-0.200pt]{0.400pt}{0.400pt}}%
\put(170.0,82.0){\rule[-0.200pt]{4.818pt}{0.400pt
…
\put(170.0,860.0){\rule[-0.200pt]{308.352pt}{0.400pt}}
\end{picture}
```

Gnuplot and LaTeX

- LaTeX document:

```
\documentclass{article}
\usepackage{geometry}
\usepackage{color}
\usepackage{graphicx}
\begin{document}
\begin{figure}
  \begin{center}
    \include{example1}
  \end{center}
  \caption{A caption}
\end{figure}
\end{document}
```

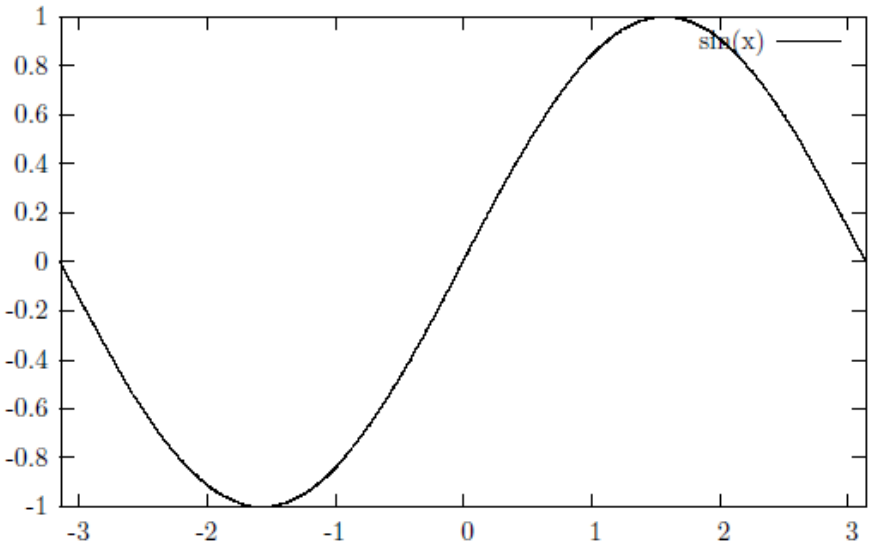


Figure 1: A caption

ImageMagick

ImageMagick (http://www.imagemagick.org) is a powerful, free software suite to create, edit, and compose bitmap images. It can read, convert and write images in a large variety of formats.

ImageMagick - Tools

- convert
  - Convert image formats.
  - Many image transformations available – scale, rotate, text, artistic filters
- display
- import
  - Screen capture under Linux
- identify
  - Find out characteristics of image
- composite
  - Composite multiple images together

ImageMagick - processing

- Add a blue border around image (expand?, replace?, shrink?)
  - `convert -border 10x10 -bordercolor "#6699ff" fan.png fan2.png`
- Add a caption/attribution
  - `convert -font helvetica -fill red -pointsize 14 -draw 'text 446,404 "made by……"' fan2.png fan3.png`

ImageMagick - movies

- ImageMagick can create animations (animated GIFs only)
  - `convert -delay 20 -loop 0 file*.png anim.gif`
  - `animate anim.gif`

ImageMagick - actions

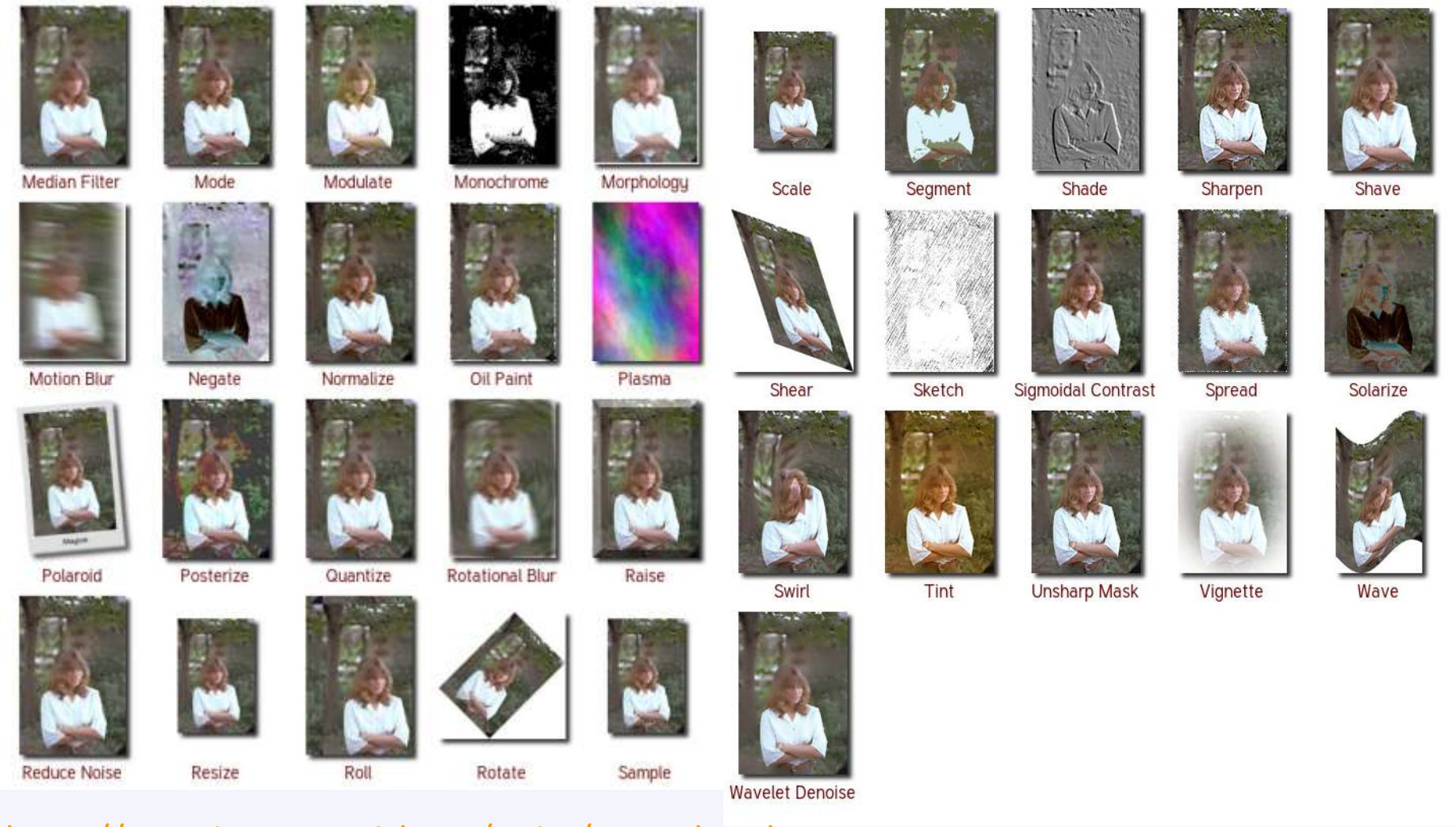
feature	action
Animation	create a GIF animation sequence from a group of images.
Color management	accurate color management with color profiles or in lieu of-- built-in gamma compression or expansion as demanded by the colorspace
Composite	overlap one image over another
Decorate	add a border or frame to an image.
Discrete Fourier Transf.	implements the forward and inverse DFT.
Draw	add shapes or text to an image.
Format conversion	convert an image from one format to another (e.g. PNG to JPEG).
Generalized pixel distortion	correct for, or induce image distortions including perspective.
Image calculator	apply a mathematical expression to an image or image channels.
Image gradients	create a gradual blend of two colors whose shape is horizontal, vertical, circular, or elliptical.
Image identification	describe the format and attributes of an image.
Transform	resize, rotate, deskew, crop, flip or trim an image.
Transparency	render portions of an image invisible.

ImageMagick - examples



https://www.imagemagick.org/script/examples.php

ImageMagick - examples



https://www.imagemagick.org/script/examples.php

ImageMagick – batch processing

Batch processing -> working with a set or sequence of images

- All images generally must be the same size
- Change colors:

```
convert -colorspace Gray example01.png example001.png
```

- Resize:

```
convert -scale 20%% example01.png example001.png
```

- Add a caption:

```
convert -size 320x85 -draw "text 25,60 'Made by....'" -fill darkred
example01.png example001.png
```

- Process all files:

```
for file in *.png ;do  convert -colorspace Gray $file
${file%.png}.jpg; done
```

Hands-on 4

1. Plot 10 png images in gnuplot. The first one should show 1 circle, next ones should add and an extra circle with increased radius.
2. Convert the images to grayscale and save them into *.jpg file
3. Make an animated gif
4. Open animation (in the web browser or file browser)

1.
```
#! /usr/bin/gnuplot
set polar
set size square
set term png
set xrange [-10:10];set yrange [-10:10]
r=1
do for [r=1:10] {
outfile = sprintf('radius%02.0f.png',r)
set output outfile
plot for [i=0:r-1] r-i
}
```
2. for file in *.png ;do convert -colorspace Gray $file ${file%.png}.jpg; done or manually:
```
convert -colorspace Gray radius01.png radius01.jpg
….
```
3.
```
convert -delay 20 *png radius.gif
```
4. …..

Questions

Helpdesk:
hpcinfo@kuleuven.be or https://admin.kuleuven.be/icts/HPCinfo_form/HPC-info-formulier

VSC web site:
http://www.vscentrum.be/

VSC documentation: https://docs.vscentrum.be

VSC agenda: training sessions, events

Systems status page:
http://status.vscentrum.be

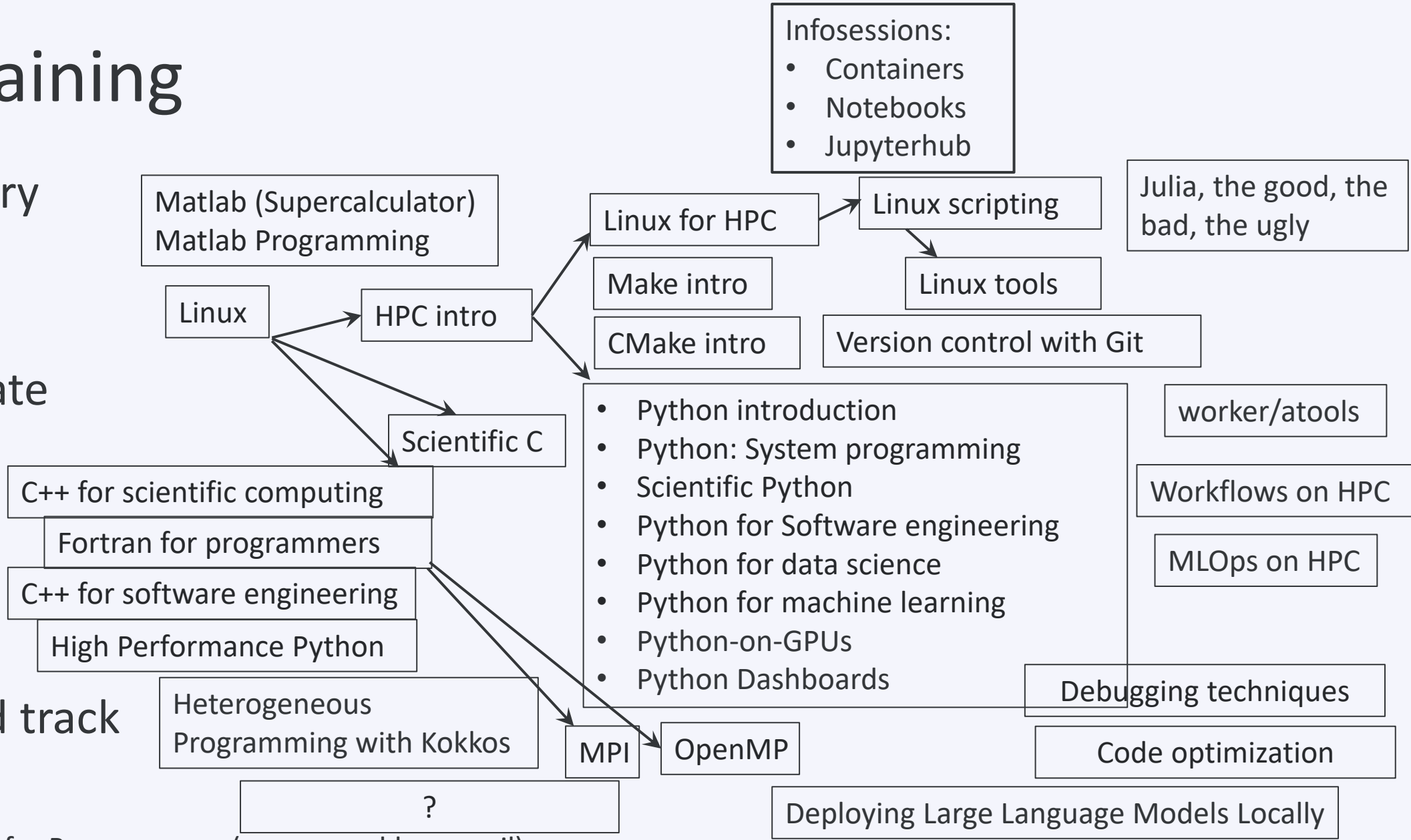
Stay Connected to VSC

LinkedIn®



VSC training

- Introductory
- Intermediate
- Advanced
- Specialized track



edX MOOC Fortran for Programmers (announced by e-mail)
edX Defensive programming and debugging